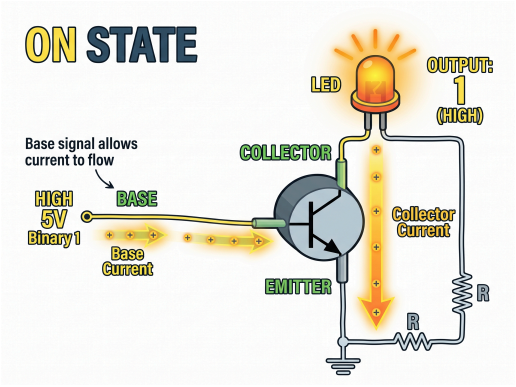
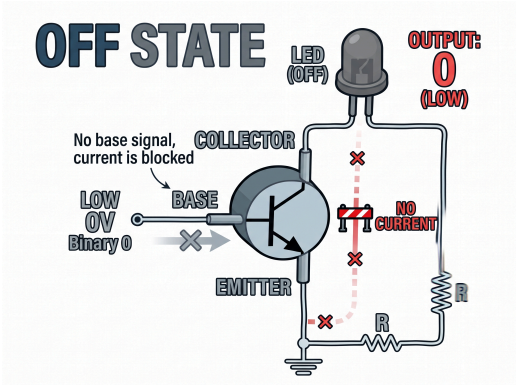
I Temel Programlama Yapıları

1. Donanımın Temeli: Transistör ve İkili Sistem (Binary)

Transistör elektrik akımını geçirerek ya da keserek 1 veya 0 değerini üreten bilgisayarın en temel donanım birimidir. İşlemci, hafıza gibi bilgisayar donanımlarında milyarlarca transistör bulunur.

Bir Transistör Elektrik Akını Geçirir veya Keser





- Elektrik Akımı Var (Açık) : 1 (Bit)
- Elektrik Akımı Yok (Kapalı) : 0 (Bit)

1 ve 0 durumunu oluşturan en küçük bilgi birimine **Bit (Binary Digit)** denir. Bilgisayar dünyasındaki tüm yazılımlar, videolar, oyunlar ve metinler fiziksel temelde sadece 1 ve 0'lardan ibarettir.

Yazılım tarihinin en başında, bilgisayarlara bir iş yaptırmak için işlemcinin tek doğal dili olan Makine Dili (**İkilik Sistem / Binary**) kullanılıyordu. Programcılar, komutları `1011 0000` gibi 0 ve 1 dizileri halinde delikli kartlara veya anahtarlara işliyordu. Ancak bu yöntem insan gözü için son derece yorucuydu ve tek bir rakamın yanlış yazılması tüm programı bozuyordu.

2. İlk Bilgisayarlar ve 8 Bitlik Komut Girişleri (Altair 8800)

Erken dönem **Altair 8800** gibi bilgisayarların klavyesi veya ekranı yoktu. Ön panelinde sıra sıra fiziksel anahtarlar (**toggle switches**) ve LED lambalar yer alıyordu. Bu bilgisayar, sadece 1 ve 0 değerleri kullanılarak donanıma nasıl komut verildiğine ve hesap yaptırıldığına somut bir örnektir.

```
+-------------------------------------------------------------+
|                  ALTAIR 8800 ÖN PANELİ                      |
|                                                             |
|   LED'ler (Çıkış):    (O) (X) (O) (O)   (X) (X) (O) (X)     |
|                        0   1   0   0     1   1   0   1      |
|                                                             |
|   Anahtarlar (Giriş): [v] [^] [v] [v]   [^] [^] [v] [^]     |
|                        0   1   0   0     1   1   0   1      |
+-------------------------------------------------------------+
```

- Programcılar her bir 8 bitlik komutu anahtarları tek tek yukarı (1) ve aşağı (0) indirerek belleğe elle yüklüyordu.
- İki sayıyı toplamak için bile onlarca 8 bitlik komutu hatasız girmek gerekiyordu. Bu süreç son derece yavaş, zihinsel olarak yorucu ve hata yapmaya açıktı.

Tarihi Eser Bilgisayarı Programladım | Altair 8800

Ömer Bektaş'ın kanalında Altair 8800 hakkında detaylı bir video bulunmakta.

www.youtube.com/watch?v=1zwTRA0yaYo

Altair 8800 Emulatorü

Altair 8800 cihazını denemek istiyorsanız tarayıcınızda kullanabileceğiniz Altair 8800 Emulator linki.

www.s2js.com/altair/sim.html

Ömer Bektaş'ın Youtube Kanalı

Ömer Bektaş, bilişim dünyasına yönelik çok faydalı içerikler üreten bir YouTube içerik üreticisidir. Bu kanalı takip etmeniz sizin için çok faydalı olacaktır.

www.youtube.com/@merbekta

3. Onaltılık Sayı Sistemi (Hexadecimal)

İkilik Sistemde oluşan zorlukları aşmak için **Onaltılık (Hexadecimal)** Sayı Sistemi kullanılmaya başlandı. Dört adet ikilik (binary) basamağın tek bir hexadecimal karaktere (0-9 ve A-F arası) karşılık gelmesi sayesinde, `1011 0000`, `0110 0001` gibi karmaşık bir makine kodu `B0`, `61` şeklinde çok daha kısa ve okunabilir bir biçimde ifade edildi. Bu yöntem yazımı kısaltıp hata payını azaltsa da, programcıların hâlâ hangi sayı kodunun hangi işlemi yaptığını ezbere bilmesi gerekiyordu.
$2^4 = 16$ olduğu için 4 bitlik bir binary sayı tam olarak bir hexadecimal basamağı karşılık gelir.

4-Bit İkilik (Binary) ve Onaltılık (Hexadecimal) Karşılıklar Tablosu

Hex	Dec	Binary	Hex	Dec	Binary	Hex	Dec	Binary	Hex	Dec	Binary
0	0	0000	**4**	4	0100	**8**	8	1000	**C**	12	1100
1	1	0001	**5**	5	0101	**9**	9	1001	**D**	13	1101
2	2	0010	**6**	6	0110	**A**	10	1010	**E**	14	1110
3	3	0011	**7**	7	0111	**B**	11	1011	**F**	15	1111

Hex: Hexadecimal(16'lık), **Dec:** Decimal(10'luk), **Binary:** Binary(4 Bit),

4. Metin Temsili: 8 Bit = 1 Byte ve ASCII Tablosu

Bir harfi veya sembolü sayılarla temsil etmek için standart referans tablosu **ASCII (American Standard Code for Information Interchange)** geliştirilmiştir. Böylelikle en temel düzeyde gerekli olan karakterlere için **ASCII** tablosundaki karakterin sayısal bir karşılığı oluşturuldu.

- Bilgisayar mimarisinin temelinde 1 karakter 1 Byte (8 bit) yer kaplar. ASCII tablosunda 256 karakter bulunmasının sebebi 8 bit'in yan yana getirilerek 1 Byte oluşturmasıdır. 1 Byte aynı zamanda iki hexadecimal karakterle de kolaylıkla ifade edilebilir.
- **Kontrol Karakterleri:** Tablodaki 0 ile 31 arasındaki ilk 32 karakter ve 127 numaralı `DEL` karakteri ekranda basılabilir bir harf veya sembol değildir. Bunlar cihazlar arası iletişimi ve imleç hareketlerini yöneten **Kontrol Karakterleri (Control / Non-printable Characters)** olarak adlandırılır.

ASCII Code - Karakter Kodlama Tablosu

Tüm ASCII karakterlerini incelemek için bu referans tablosunu kullanabilirsiniz.

www.ascii-code.com

Temel ASCII Tablosu (32-127 Karakterin Kodu)

Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr
32	20	0010 0000	*SP*	56	38	0011 1000	8	80	50	0101 0000	P	104	68	0110 1000	h
33	21	0010 0001	!	57	39	0011 1001	9	81	51	0101 0001	Q	105	69	0110 1001	i
34	22	0010 0010	"	58	3A	0011 1010	:	82	52	0101 0010	R	106	6A	0110 1010	j
35	23	0010 0011	#	59	3B	0011 1011	;	83	53	0101 0011	S	107	6B	0110 1011	k
36	24	0010 0100	$	60	3C	0011 1100	<	84	54	0101 0100	T	108	6C	0110 1100	l
37	25	0010 0101	%	61	3D	0011 1101	=	85	55	0101 0101	U	109	6D	0110 1101	m
38	26	0010 0110	&	62	3E	0011 1110	>	86	56	0101 0110	V	110	6E	0110 1110	n
39	27	0010 0111	'	63	3F	0011 1111	?	87	57	0101 0111	W	111	6F	0110 1111	o
40	28	0010 1000	(	64	40	0100 0000	@	88	58	0101 1000	X	112	70	0111 0000	p
41	29	0010 1001	)	65	41	0100 0001	A	89	59	0101 1001	Y	113	71	0111 0001	q
42	2A	0010 1010	*	66	42	0100 0010	B	90	5A	0101 1010	Z	114	72	0111 0010	r
43	2B	0010 1011	+	67	43	0100 0011	C	91	5B	0101 1011	[	115	73	0111 0011	s
44	2C	0010 1100	,	68	44	0100 0100	D	92	5C	0101 1100	\	116	74	0111 0100	t
45	2D	0010 1101	-	69	45	0100 0101	E	93	5D	0101 1101	]	117	75	0111 0101	u
46	2E	0010 1110	.	70	46	0100 0110	F	94	5E	0101 1110	^	118	76	0111 0110	v
47	2F	0010 1111	/	71	47	0100 0111	G	95	5F	0101 1111	_	119	77	0111 0111	w
48	30	0011 0000	0	72	48	0100 1000	H	96	60	0110 0000	`	120	78	0111 1000	x
49	31	0011 0001	1	73	49	0100 1001	I	97	61	0110 0001	a	121	79	0111 1001	y
50	32	0011 0010	2	74	4A	0100 1010	J	98	62	0110 0010	b	122	7A	0111 1010	z
51	33	0011 0011	3	75	4B	0100 1011	K	99	63	0110 0011	c	123	7B	0111 1011	{
52	34	0011 0100	4	76	4C	0100 1100	L	100	64	0110 0100	d	124	7C	0111 1100	|
53	35	0011 0101	5	77	4D	0100 1101	M	101	65	0110 0101	e	125	7D	0111 1101	}
54	36	0011 0110	6	78	4E	0100 1110	N	102	66	0110 0110	f	126	7E	0111 1110	~
55	37	0011 0111	7	79	4F	0100 1111	O	103	67	0110 0111	g	127	7F	0111 1111	*DEL*

5. Bit'ten Terabayt'a Veri Büyüklükleri

Bilgisayar dünyasındaki tüm içeriklerin fiziksel temelde sadece 1 ve 0'lardan ibaret olduğunu ve 1 ve 0 durumun oluşturan en küçük bilgi birimine **Bit (Binary Digit)** dendiğini söylemiştik.

- **Bit (Binary Digit):** Bilgisayardaki en küçük mantıksal veri birimidir. Sadece **0** veya **1** değerini alabilir. Devredeki tek bir transistör lamba anahtarı gibi düşünülebilir: ya kapalıdır (0) ya da açıktır (1).
- **Byte:** 8 adet bitin yan yana gelmesiyle oluşan veri kümesidir. $8 \text{ Bit} = 1 \text{ Byte}$

Örneğin **"A"** harfi bellekte **1 Byte** yer kaplar gibi düşünebiliriz.

- Günlük hayatta metrik sistemde birimler onluk sisteme (10^3) göre biner biner büyür (1 kg = 1000 g, 1 km = 1000 m). Ancak bilgisayarlar **2'lik sayı sistemine (Binary)** göre çalışır. Bilgisayar mimarisinde bellek adreslemesi 2'nin üsleri şeklinde katlanarak ilerler. Onluk sistemdeki 1000 sayısına ikili tabanda en yakın kuvvet: 1024 sayısıdır ($2^{10} = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1024$).
  Bu nedenle bilişim mimarisinde her bir büyüklük basamağı bir öncekinin **1024 katı** (2^{10}) kabul edilir.

Veri Büyüklüğü Basamakları

Birim	Kapasite	Kısaltma	Somut Karşılığı
Bit	0 veya 1	**b**	Tek bir transistör / elektrik sinyali
Byte	8 Bit	**B**	Tek bir karakter (harf, rakam, sembol)
Kilobyte	1024 Byte	**KB**	Düz metinden oluşan 1 sayfalık belge
Megabyte	1024 KB	**MB**	1 dakikalık MP3 ses kaydı veya telefon fotoğrafı
Gigabyte	1024 MB	**GB**	Yaklaşık 1 saatlik HD kalitede film
Terabyte	1024 GB	**TB**	Geniş harici yedekleme diski, çok sayıda büyük oyun
Petabyte	1024 TB	**PB**	Büyük veri merkezleri veya bulut sunucu havuzları
Exabyte	1024 PB	**EB**	Küresel internet trafiğinin aylık ölçekteki hacmi

Donanım Üreticisi vs. İşletim Sistemi Farkı: Piyasadan satın alınan **1TB** kapasiteli bir SSD veya harici disk bilgisayara takıldığında, depolama alanı genellikle yaklaşık **931 GB** görünür. Bu durum donanım hatası değildir:

Donanım Üreticileri **Onluk** sayı birimini kullanarak $1 \ \ \mathrm{KB} = 10^3 \ \ \mathrm{Byte}$ şeklinde hesap yapar:

$$1 \ \ \mathrm{TB} = 1.000.000.000.000 \ \ \mathrm{Byte}$$

Fakat işletim sistemleri ikilik sayı sistemine göre ölçüm yaptığı için altta ki bölme işleminin sonucu satın aldığınız diskinizin gerçek boyutu **1TB** altında olacaktır.

$$\frac{1.000.000.000.000}{1024 \times 1024 \times 1024} \approx 931.32 \ \ \mathrm{GB}$$

Sizce bu disk üreticilerinin yaptığı bir hile midir?

6. Makine Kodundan Assembly Diline

Altair 8800 gibi erken dönem bilgisayarlarda, işlemciye komut vermek için sürekli **`1` ve `0`** dizilimlerinden oluşan makine kodlarını kullanmak oldukça zahmetliydi ve hatalara açıktı. Makine kodlarını daha kısa ve okunabilir biçimde ifade etmek için **Onaltılık (Hexadecimal) sayı sistemi** kullanılmaya başlandı. Ancak hexadecimal gösterim, makine kodunu daha kolay yazmayı sağlasa da komutların anlamını doğrudan açıklamıyordu.

Bu nedenle makine kodundaki komutları daha kolay hatırlayabilmek ve yazabilmek için İngilizce kelimelerin kısaltmalarından oluşan sembolik isimler (**Mnemonic**) kullanılmaya başlandı. Bu sembolik komutları kullanan dile **Assembly dili** adı verildi. Böylece programcılar, yalnızca **`1` ve `0`** dizilimleriyle uğraşmak yerine, daha anlaşılır ve hatırlanması kolay komutlarla program yazabilir hâle geldi.

Hexadecimal Yerine Assembly Kullanımı Örneği

Hexadecimal	Yerine	Assembly	Açıklama
`B0 61`	⟶	`MOV AL, 61H`	AL yazmacına **`61H`** verisi taşındı.
`04 05`	⟶	`ADD AL, 05H`	AL yazmacındaki değere **`05H`** eklendi.
`2C 02`	⟶	`SUB AL, 02H`	AL yazmacındaki değerden **`02H`** çıkarıldı.

Bu sembolik komutlarla yazılan programlar, **Assembler (Çevirici)** adı verilen bir program aracılığıyla işlemcinin anlayabileceği makine koduna dönüştürüldü.

x86 Assembly: İki Sayıyı Toplama Örneği

```
section .text
global _start

_start:
    mov eax, 5          ; ilk sayı
    mov ebx, 7          ; ikinci sayı
    add eax, ebx        ; eax = eax + ebx  (5 + 7 = 12)

    mov ebx, eax        ; sonucu ebx'e al (exit kodu ebx'ten okunur)
    mov eax, 1          ; sys_exit sistem çağrısı numarası
    int 0x80            ; çekirdeği çağır
```

Dillerin Atası | Assembly

Ömer Bektaş'ın kanalında Assembly dili hakkında detaylı bir video bulunmakta.

www.youtube.com/watch?v=yWXuG3RLkTE

7. Programlama Dilleri

Her ne kadar `1` ve `0` veya **hexadecimal sayılar** ile kodlama yapmaya göre **Assembly** dili büyük bir kolaylık sağlasa da projeler büyüdükçe kodları yazmak zorlaşıyordu. Ayrıca Assembly kodları işlemciden işlemciye farklılık gösteriyor, bu da her yeni işlemci için kodların yeniden yazılması zorunluluğunu doğuruyordu. Bu duruma **donanım bağımlılığı** deniyordu.

İşte bu sorunu çözmek için **Fortran, COBOL ve C** gibi programlama dilleri ortaya çıktı. Artık iki sayıyı toplamak için işlemci yazmaçlarıyla uğraşmak yerine doğrudan `sonuc = a + b` yazmak yeterli hale geldi. Programlama dilleri ile yazılan kodlar, **Derleyici (Compiler)** veya **Yorumlayıcı (Interpreter)** adı verilen araçlarla makine koduna çevrilerek işlemci üzerinde çalıştırılmaktadır. Ayrıca programlama dili ile yazılan tek bir kodun hedef bilgisayarda derlenmesiyle **donanım bağımlılığı** sorunu çözüldü ve aynı kodların farklı işlemci mimarilerinde çalışabilmesi sağlandı.

Derleyici (Compiler): Programcının yazdığı kodun tamamını önce baştan sona okur, yazım hatalarını kontrol eder ve tüm kodu bir bütün olarak makine diline çevirir. Kodun yazımında (söz diziminde) bir hata varsa derleme işlemi tamamlanmaz.

Yorumlayıcı (Interpreter): Kodu bir bütün olarak değil satır satır okuyarak anında çalıştırır. Bir satır çevrilip çalıştırıldıktan sonra sıradaki satıra geçilir. Hata tespiti açısından yorumlayıcı, hatanın bulunduğu satıra gelene kadar programı çalıştırır ve o noktada durur. Bu yaklaşım tıpkı bir tercümanın, konuşmacının cümlelerini anlık olarak çevirmesine benzer. **Python ve Ruby**, bu yaklaşımı kullanan dillere örnek gösterilebilir.

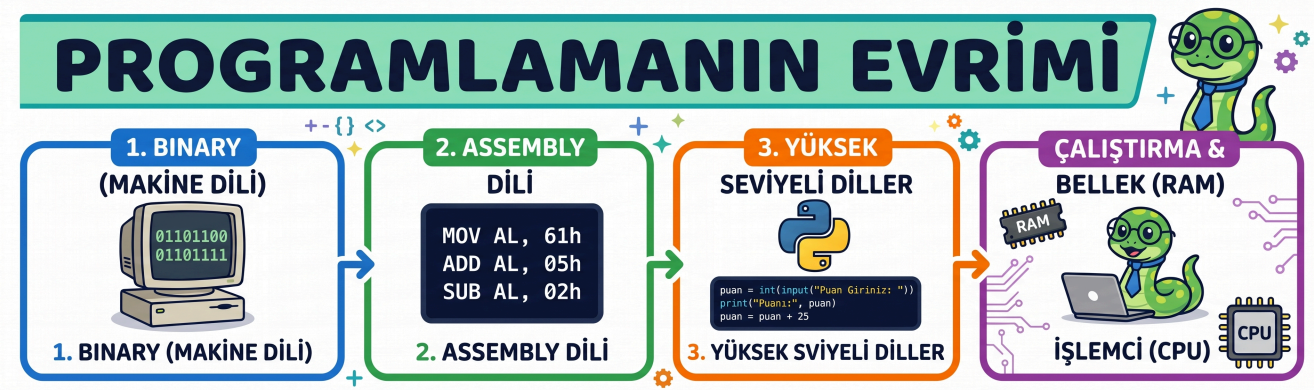
Evrim Aşaması	Temsil Biçimi	Örnek Kod	Temel Avantajı	Temel Zorluğu
1. Makine Dili (Binary)	`0` ve `1` dizileri	`1011 0000` `0110 0001`	İşlemci hiçbir çeviriye ihtiyaç duymadan doğrudan çalıştırır.	İnsan tarafından okunması ve yazılması çok zordur.
2. Hexadecimal (16'lık Sistem)	`0-9` ve `A-F` karakterleri	`B0 61`	Uzun 0 ve 1 dizilerini 4'lü gruplar halinde kısaltarak yazım kolaylığı sağlar.	Komutların sayısal karşılıklarını akılda tutmak gerekiyordu.
3. Assembly Dili (Sembolik Dil)	Sembolik kısaltmalar *(Mnemonics)*	`MOV AL, 61h`	Komutlar (`MOV`, `ADD`, `SUB`) anlamlı kelimelerle ifade edilir.	Her işlemci mimarisi için farklıdır; büyük projelerde kod çok uzar.
4. Yüksek Seviyeli Diller	Günlük dil ve matematiksel ifadeler	`int a = 97;`	Öğrenmesi kolaydır, donanımdan bağımsızdır ve hızlı yazılım geliştirmeyi sağlar.	İşlemcide çalışabilmesi için derleyici veya yorumlayıcı ile çevrilmesi gerekir.

Örnek Python Kodu

```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```



Bilgisayarın Dili

Transistörden yüksek seviyeli programlama dillerine: 1 ve 0'ın yolculuğu

Programlamanın Evrimi

1 Makine Dili

`1011 0000`

✓ İşlemci çeviri olmadan çalıştırır
✗ Okuması ve yazması çok zor

2 Hexadecimal

`B0 61`

✓ 0/1 dizilerini 4'lü gruplarla kısaltır
✗ Komut kodları ezberlenmeli

3 Assembly

`MOV AL, 61H`

✓ Anlamlı kısaltmalar (MOV, ADD)
✗ Her işlemciye özel, kod uzar

4 Yüksek Seviyeli

`sonuc = a + b`

✓ Kolay, donanımdan bağımsız, hızlı
✗ Derleyici/yorumlayıcı gerekir

1 · Transistör: Akımı Geçirir veya Keser

Bilgisayarın en temel donanım birimi; işlemci ve hafızada **milyarlarca** transistör bulunur.

AÇIK · Çıkış 1
Base: HIGH (5V)
Akım geçer, LED yanar

KAPALI · Çıkış 0
Base: LOW (0V)
Akım kesilir, LED söner

2 · Bit (Binary Digit)

1 ve 0 durumunu oluşturan **en küçük bilgi birimi.** Yazılımlar, videolar, oyunlar ve metinler fiziksel olarak yalnızca 1 ve 0'lardan oluşur.

1 0 1 1

3 · Altair 8800 (1970'lerin ortası)

Klavye ve ekran yoktu; ön panelde **toggle switch** ve LED'ler vardı. Her 8 bitlik komut anahtarlarla tek tek belleğe yükleniyordu: yukarı = 1, aşağı = 0.

```
+-------------------------------------------------------------+
|                  ALTAIR 8800 ÖN PANELİ                      |
|                                                             |
|   LED'ler (Çıkış):    (O) (X) (O) (O)   (X) (X) (O) (X)     |
|                        0   1   0   0     1   1   0   1      |
|                                                             |
|   Anahtarlar (Giriş): [v] [^] [v] [v]   [^] [^] [v] [^]     |
|                        0   1   0   0     1   1   0   1      |
+-------------------------------------------------------------+
```

İki sayıyı toplamak için bile onlarca komut hatasız girilmeliydi: yavaş, yorucu, hataya açık.

4 · Hexadecimal: 4 Bit = 1 Basamak

$2^4 = 16$ olduğundan 4 bit, tek bir hex karakterine (0–9, A–F) karşılık gelir.

0 (0) 0000	**1** (1) 0001	**2** (2) 0010	**3** (3) 0011
4 (4) 0100	**5** (5) 0101	**6** (6) 0110	**7** (7) 0111
8 (8) 1000	**9** (9) 1001	**A** (10) 1010	**B** (11) 1011
C (12) 1100	**D** (13) 1101	**E** (14) 1110	**F** (15) 1111

`1011 0000` → **B0** `0110 0001` → **61**

5 · Metin Temsili: 8 Bit = 1 Byte ve ASCII

Harf ve sembolleri sayılarla temsil etmek için **ASCII** standardı geliştirildi. 1 karakter = **1 Byte** (8 bit) yer kaplar ve iki hex karakterle yazılır.

0 1 0 0 0 0 0 1

1 Byte = hex **41** = onluk **65** = '**A**'

Chr	Dec	Hex	Binary
SP	32	20	0010 0000
0	48	30	0011 0000
A	65	41	0100 0001
a	97	61	0110 0001
DEL	127	7F	0111 1111

0–31 ve 127	32–47	48–57	65–90	97–122
Kontrol karakterleri (basılamaz)	Boşluk ve semboller	Rakamlar 0–9	Büyük harfler A–Z	Küçük harfler a–z

6 · Makine Kodundan Assembly'ye

Komutlar, İngilizce kelime kısaltmalarından oluşan sembolik isimlerle (**mnemonic**) yazıldı. **Assembler** bu kodu makine koduna çevirir.

Hex	Assembly	Açıklama
B0 61	MOV AL, 61H	AL'ye 61H taşındı
04 05	ADD AL, 05H	AL'ye 05H eklendi
2C 02	SUB AL, 02H	AL'den 02H çıkarıldı

x86 · 5 + 7

```
mov eax, 5 ; ilk sayı
mov ebx, 7 ; ikinci sayı
add eax, ebx ; 12
```

Yüksek seviyede: `sonuc = a + b`

7 · Programlama Dilleri: Derleyici ve Yorumlayıcı

Fortran, COBOL ve C gibi dillerde çıktı; makine koduna çevrilir. Öğrenmesi kolaydır, donanımdan bağımsızdır ve hızlı yazılım geliştirmeyi sağlar.

Derleyici (Compiler)

Kodun **tamamını** baştan sona okur, hataları denetler, bütünü makine diline çevirir. Söz diziminde hata varsa derleme tamamlanmaz.

Yorumlayıcı (Interpreter)

Kodu **satır satır** okuyup anında çalıştırır; hatalı satıra gelince durur. Tıpkı anlık çeviri yapan bir tercüman gibi (Python, Ruby).

PYTHON · YORUMLAYICI ÖRNEĞİ

```
1 puan = int(input("Puan Giriniz: "))
2 print("Puanı:", puan)
3 puan = puan + 25
4 print("Güncellenmiş Puan:", puan)
```

Satırlar sırayla yorumlanıp çalıştırılır.

8. Değişkenler ve Bellek(RAM)

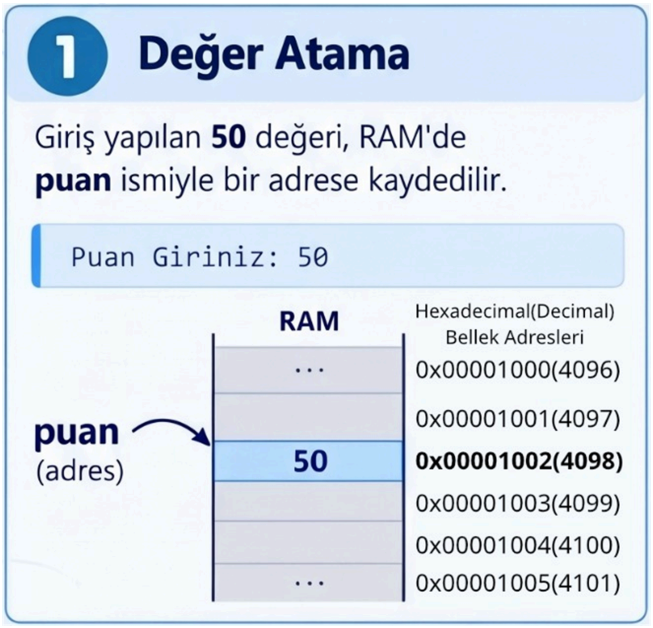
Değişken; programın çalışması sırasında verilerin bellekte (RAM) geçici olarak tutulduğu alanlara verilen isimdir. Kullanıcıdan veya harici bir kaynaktan alınan bilgiler bu değişkenlerde saklanır; kod çalıştıkça hesaplamalarda kullanılır ve süreç içinde yeni değerler alabilir.
Değişkenler yardımıyla bellekte (RAM) saklanan verilere kolayca erişilir; kod çalıştırıldığında bu değişken isimleri çağrılarak temsil ettikleri veriler üzerinde işlemler gerçekleştirilir.

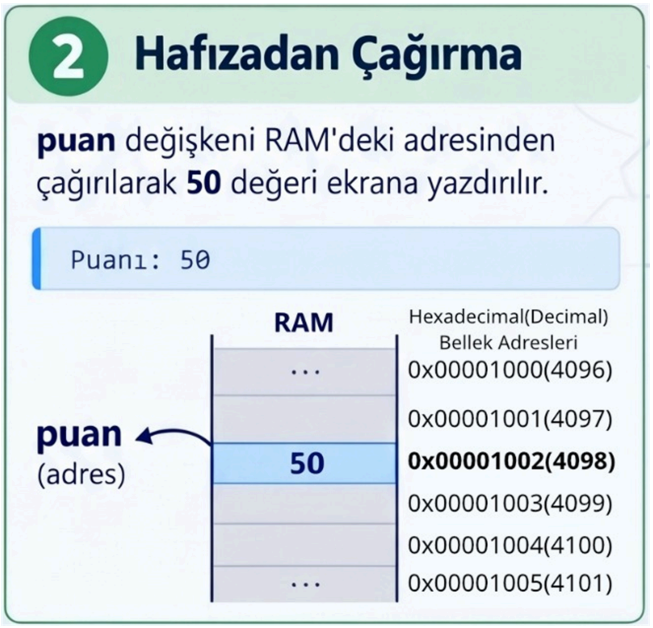
Örnek Python Kodu

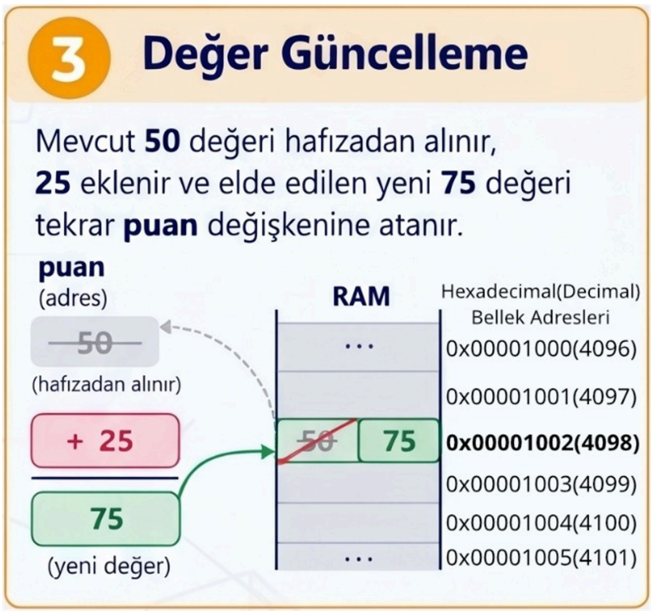
```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

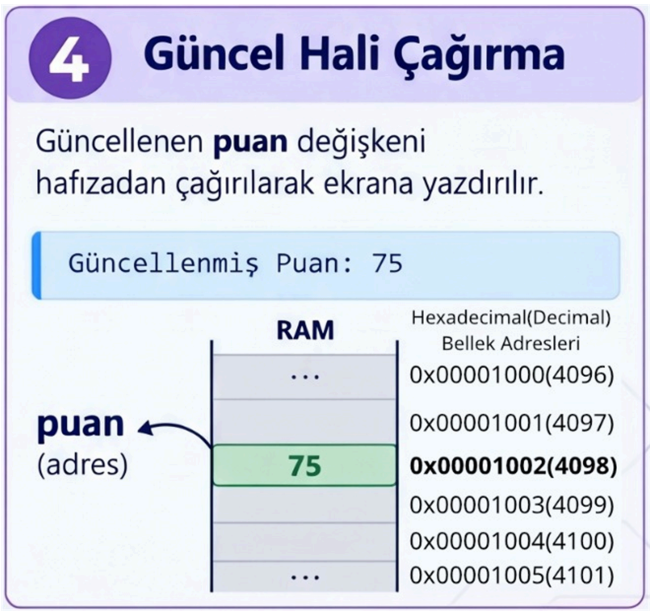
Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```







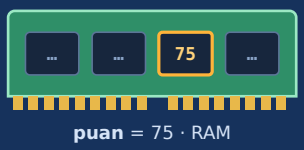


Kısacası: Değişkenler, değerleri geçici olarak RAM'de tutan etiketlerdir. Program çalıştığı sürece bu değerler kullanılabilir ve güncellenebilir.

x = 50
RAM

Değişkenler ve Bellek

Programda tanımlanan değişkenler, geçici olarak bellekte (RAM) bir alan tutar ve bu alanlara değerler kaydedilir.



puan = 75 · RAM

Python Kodu

```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Kullanılan Değişken

puan

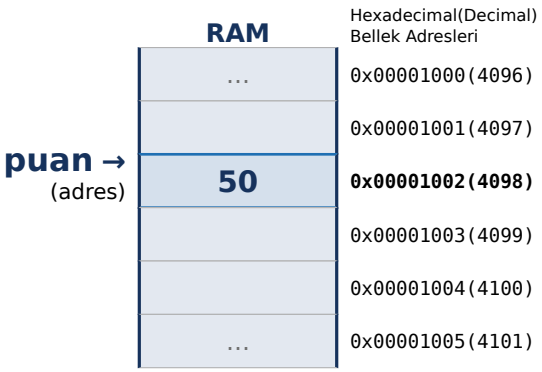
(tamsayı / int)

Puan değerini geçici olarak tutan değişken.

1 Değer Atama

Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

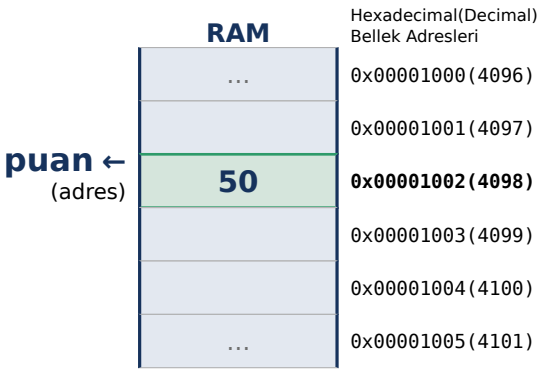
```
Puan Giriniz: 50
```



2 Hafızadan Çağırma

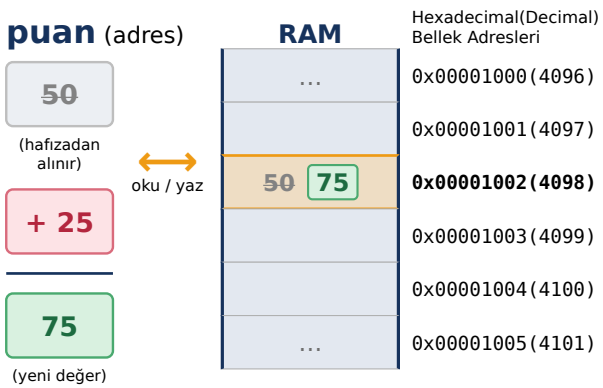
puan değişkeni RAM'deki adresinden çağırılarak **50** değeri ekrana yazdırılır.

```
Puanı: 50
```



3 Değer Güncelleme

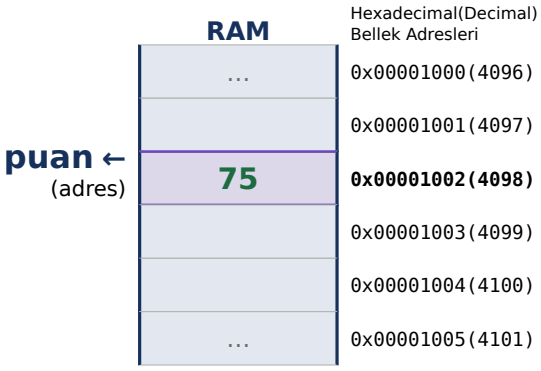
Mevcut **50** değeri hafızadan alınır, **25** eklenir ve elde edilen yeni **75** değeri tekrar **puan** değişkenine atanır.



4 Güncel Hali Çağırma

Güncellenen **puan** değişkeni hafızadan çağırılarak ekrana yazdırılır.

```
Güncellenmiş Puan: 75
```



Kısaca: Değişkenler, değerleri geçici olarak RAM'de tutan etiketlerdir. Program çalıştığı sürece bu değerler kullanılabilir ve güncellenebilir.

9. Veri Türleri ve RAM Kullanımı

Bilgisayar programları verileri RAM (bellek) üzerinde saklar. RAM'i, üst üste dizilmiş çok sayıda küçük kutu gibi düşünebiliriz. Her kutunun boyutu 1 bayt'tır (1 bayt = 8 bit; bit yalnızca 0 ya da 1 tutabilir).

- Bu veri **ne tür** bir veri? (sayı mı, karakter mi, doğru/yanlış mı?)
- Bu yüzden RAM'de **kaç kutu (bayt)** ayrılmalı ve bu veriyle **hangi işlemler** yapılabilir?

> Temel veri türleri konusunu anlatmak için C++ dili ve 16-bit sistemler (Arduino Uno, Turbo C) esas alınmıştır. Bu tercih, veri türlerinin donanımsal mantığının daha rahat anlaşılması amacıyla yapılmıştır. Derslerimizde ise Python programlama dili kullanılacak olup, Python'da veri türleri ve bellek yapısı farklı şekilde işlemektedir.

9.1. Boolean (bool): doğru / yanlış

- RAM'de en az yer kaplayan veri türüdür: **1 bayt**.
- Yalnızca iki değer alabilir: **true** (doğru, 1) veya **false** (yanlış, 0).
- Karar verme (if, while) işlemlerinde kullanılır.

```
bool kapiAcik = true;
bool sinavGecti = false;
bool girisYapti = true;
```

bool — 1 byte

```
bool kapiAcik = true;
```

1. byte	0	0	0	0	0	0	0	**1**

true değeri 1, olarak saklanır.

bool — 1 byte

```
bool sinavGecti = false;
```

1. byte	0	0	0	0	0	0	0	0

false değeri 0 olarak saklanır.

> Aslında doğru/yanlış bilgisi tek bir bit ile saklanabilir; ancak RAM'e ulaşım bayt bayt yapıldığı için bilgisayar bool için de 1 tam bayt ayırır.

9.2. Karakter (char)

- RAM'de 1 bayt yer kaplar.
- Tek bir karakter tutar: harf, rakam karakteri, noktalama işareti.
- Tek tırnak içinde yazılır: 'A'.
- Bilgisayar karakterleri ASCII tablosundaki sayı karşılığı ile saklar (örneğin 'A' karakteri 65).
- 1 bayt ($8\text{bit} = 2^{16} = 256$), ASCII tablosundaki 256 karakter için yeterlidir.
- Karakterler metin için kullanılır, matematiksel işlemlere girmez. '5' karakteri ile 5 sayısı aynı şey değildir.

```
char harf = 'A';
char rakamKarakteri = '5'; // bu bir sayı değil, karakterdir
char cinsiyet = 'E'; // 'E' (Erkek) veya 'K' (Kadın) gibi tek harfli kısaltmalar
char paraBirimi = '$'; // Özel semboller ve işaretler
char islemTuru = '+'; // Matematiksel operatör sembolleri
char cevapSikki = 'C'; // Çoktan seçmeli testlerde işaretlenen şık
```

char — 1 byte

```
char harf = 'A';
```

1. byte	0	**1**	0	0	0	0	0	**1**

'A' karakterinin ASCII kodu 65'tir.

char — 1 byte

```
char karakter = '5';
```

1. byte	0	0	**1**	**1**	0	**1**	0	**1**

'5' karakterinin ASCII kodu 53'tür Bellekte 1 byte kaplar.

9.3. Tam Sayı (int)

- RAM'de **2 bayt** (16 bit) yer kaplar.
- 16 bit ile $2^{16} = 65536$ farklı değer saklanabilir.
- Yalnızca tam sayıları tutar.
- Toplama, çıkarma, çarpma, bölme ve mod(kalan) işlemlerinde kullanılır.

```c
int a = 7;
int b = 2;
int toplam = a + b // toplam = 9
int sonuc = a / b; // sonuc = 3 (3.5 değil!)
int kalan = a % b; // kalan = 1
```

int — **2 byte**

```c
int puan = 50;
```

1. byte	0	0	0	0	0	0	0	0
2. byte	0	0	**1**	**1**	0	0	**1**	0

50 = 32 + 16 + 2 olduğu için son byte 00110010 olur. İlk byte'ın tamamı 0'dır.

int — **2 byte**

```c
int sayi = 12377;
```

1. byte	0	0	**1**	**1**	0	0	0	0
2. byte	0	**1**	0	**1**	**1**	0	0	**1**

12377 = 8192 + 4096 + 64 + 16 + 8 + 1 olduğu için ilk byte 00110000 (12288), son byte ise 01011001 (89) olur.

9.4. Ondalıklı Sayı (float)

- RAM'de **4 bayt** yer kaplar; **int**'ten 2 kat fazla alan.
- Virgüllü sayıları tutar. Kodda nokta ile yazılır: 3.14.
- Tam sayı ve küsurat kısımlarını ayrı işler.
- Küsuratlı ve geniş aralıklı sayılarda kullanılır.
- Tüm matematiksel işlemlerde kullanılabilir.

```c
float a = 7;
float b = 2;
float sonuc = a / b; // => 3.5
float pi = 3.14;
float alan = pi * b * b; // => 12.56
```

float — **4 byte**

```c
float pi = 3.14;
```

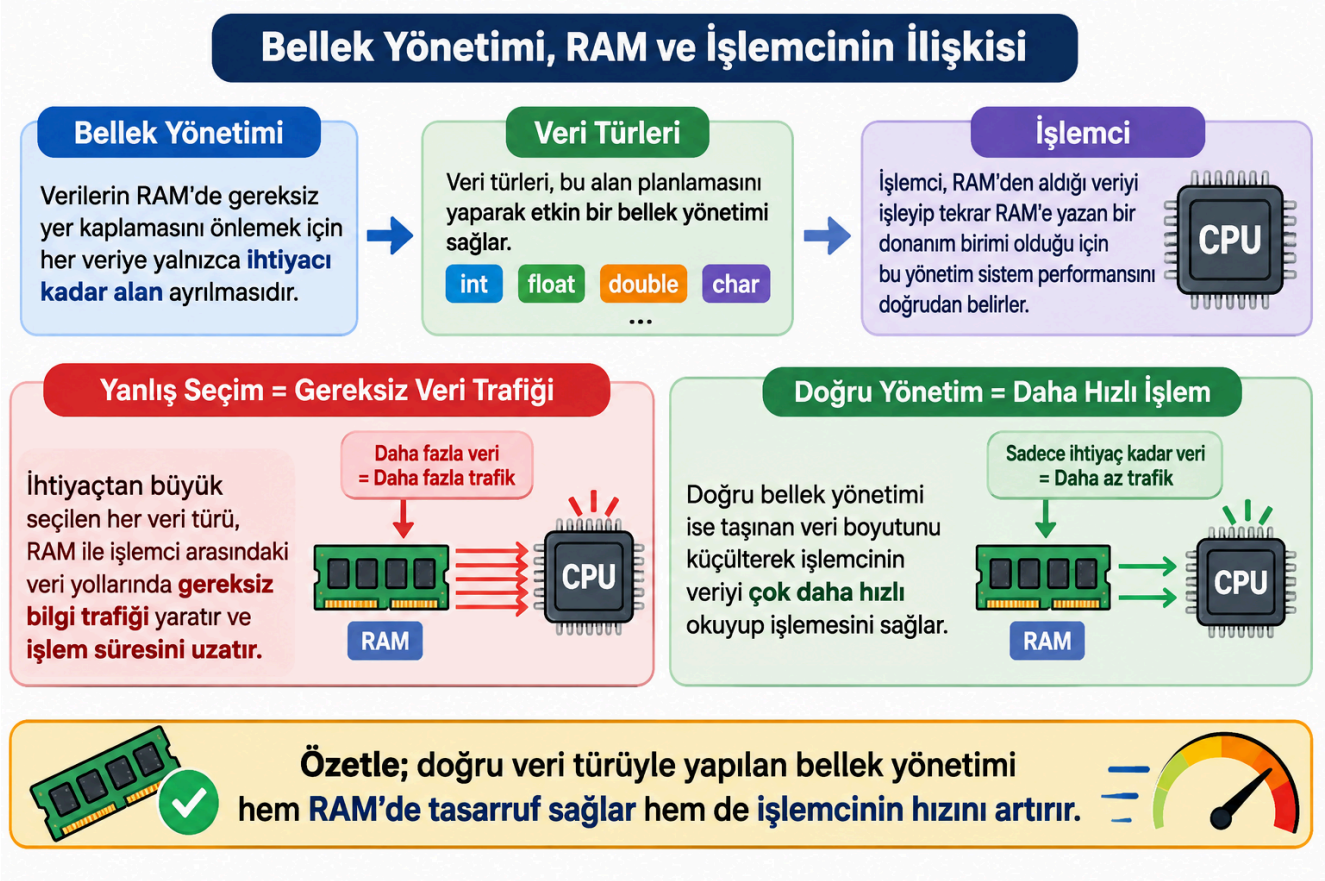
1. byte	0	**1**	0	0	0	0	0	0
2. byte	0	**1**	0	0	**1**	0	0	0
3. byte	**1**	**1**	**1**	**1**	0	**1**	0	**1**
4. byte	**1**	**1**	0	0	0	0	**1**	**1**

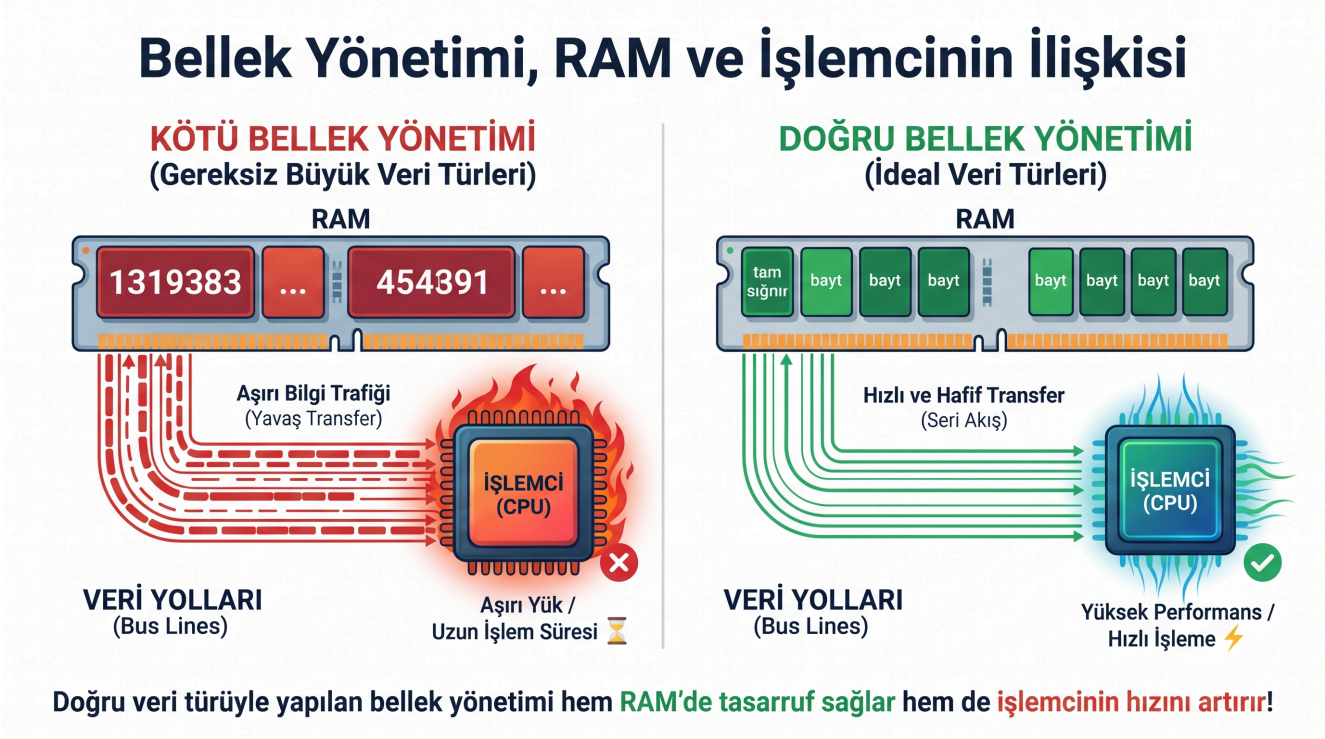
Ondalıklı sayılar IEEE 754 biçiminde saklanır ve 4 byte yer kaplar.

Ne zaman int, ne zaman float?
float her işlemde kullanılabilir; ancak hem daha fazla RAM kaplar hem de işlemciyi daha çok yorar. Bu yüzden **gerekmedikçe int** yerine **float** kullanılmamalıdır. Küsurat gerekiyorsa (ortalama, not, fiiyat, sıcaklık gibi) float tercih edilir.

10. Bellek Yönetimi, RAM ve İşlemcinin İlişkisi

Bellek yönetimi, verilerin RAM'de gereksiz yer kaplamasını önlemek için her veriye yalnızca ihtiyacı kadar alan ayrılmasıdır. Veri türleri, bu alan planlamasını yaparak etkin bir bellek yönetimi sağlar. İşlemci, RAM'den aldığı veriyi işleyip tekrar RAM'e yazan bir donanım birimi olduğu için bu yönetim sistem performansını doğrudan belirler. İhtiyaçtan büyük seçilen her veri türü, RAM ile işlemci arasındaki veri yollarında gereksiz bilgi trafiği yaratır ve işlem süresini uzatır. Doğru bellek yönetimi ise taşınan veri boyutunu küçülterek işlemcinin veriyi çok daha hızlı okuyup işlemesini sağlar. **Özetle; doğru veri türüyle yapılan bellek yönetimi hem RAM'de tasarruf sağlar hem de işlemcinin hızını artırır.**





11. Karakterden Metne (Karakter Dizisi)

Tek bir **char** verisi bellekte yalnızca **1 Bayt** yer kaplar ve tek başına sadece bir karakteri tutabilir. Birden fazla karakterden oluşan bir kelimeyi veya metni bellekte saklamak istediğimizde ise tekil karakterler yan yana getirilerek bir **karakter dizisi** oluşturulur. Programlamada aynı türdeki verilerin bellekte peş peşe sıralanarak tek bir grup halinde tutulmasına **dizi (array)** adı verilir. Metinler de aslında karakterlerin yan yana dizildiği birer dizidir; bu nedenle aşağıdaki yapı ile her bir harf için RAM üzerinde art arda gelen (bitişik) 1 baytlık çekmeceler tahsis edilir.

> **Toplam Bellek Alanı = Karakter Sayısı x 1 Byte**

KEMAL Metni RAM üzerinde nasıl saklanır?

char — **5 byte**

```
char isim[5] = {'K', 'E', 'M', 'A', 'L'};
```

1. byte	0	**1**	0	0	**1**	0	**1**	**1**
2. byte	0	**1**	0	0	0	**1**	0	**1**
3. byte	0	**1**	0	0	**1**	**1**	0	**1**
4. byte	0	**1**	0	0	0	0	0	**1**
5. byte	0	**1**	0	0	**1**	**1**	0	0

Her bir harf ASCII karşılığı ile bellekte 1 byte yer kaplar (K:75, E:69, M:77, A:65, L:76). Toplam 5 karakter = 5 byte.

Ada Lovelace Metni RAM üzerinde nasıl saklanır?

char — **12 byte**

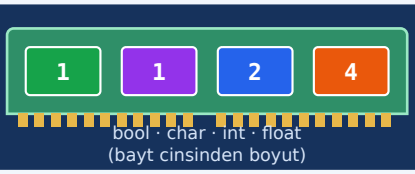
```
char tam_isim[12] = {'A', 'd', 'a', ' ', 'L', 'o', 'v', 'e', 'l', 'a', 'c', 'e'};
```

1. byte	0	**1**	0	0	0	0	0	**1**
2. byte	0	**1**	**1**	0	0	**1**	0	0
3. byte	0	**1**	**1**	0	0	0	0	**1**
4. byte	0	0	**1**	0	0	0	0	0
5. byte	0	**1**	0	0	**1**	**1**	0	0
6. byte	0	**1**	**1**	0	**1**	**1**	**1**	**1**
7. byte	0	**1**	**1**	**1**	0	**1**	**1**	0
8. byte	0	**1**	**1**	0	0	**1**	0	**1**
9. byte	0	**1**	**1**	0	**1**	**1**	0	0
10. byte	0	**1**	**1**	0	0	0	0	**1**
11. byte	0	**1**	**1**	0	0	0	**1**	**1**
12. byte	0	**1**	**1**	0	0	**1**	0	**1**

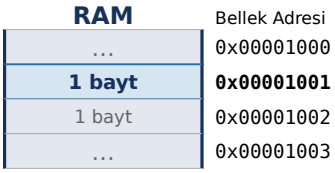
Boşluk karakteri (' ') de bellekte 1 byte yer kaplar (ASCII: 32). Toplam 12 karakter = 12 byte.

Veri Türleri ve RAM Kullanımı

Programlar verileri RAM'de bayt kutularında saklar. Verinin türü, kaç bayt yer ayrılacağını ve hangi işlemlerin yapılabileceğini belirler.



RAM: üst üste dizilmiş küçük kutular



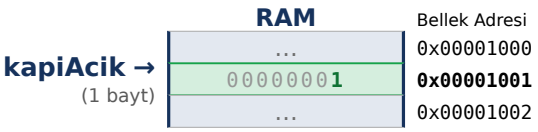
1 bayt = 8 bit
(her bit yalnızca 0 ya da 1)

1. Bu veri **ne tür** bir veri?
   (sayı mı, karakter mi, doğru/yanlış mı?)
2. RAM'de **kaç kutu (bayt)** ayrılmalı ve **hangi işlemler** yapılabilir?

bool — 1 bayt

- İki değer: **true** (1) ya da **false** (0)
- Karar yapılarında kullanılır (if, while)
- Tek bit yeter; yine de 1 tam bayt ayrılır

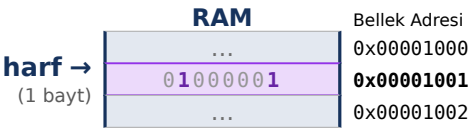
```
bool kapiAcik = true;
```



char — 1 bayt

- Tek karakter: harf, rakam karakteri, noktalama
- ASCII sayısıyla saklanır: 'A' = 65
- Matematiksel işlem yapılmaz: '5' ≠ 5

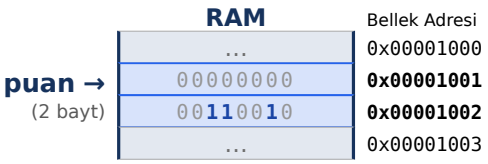
```
char harf = 'A';
```



int — 2 bayt

- 2^{16} = 65536 değer: −32768 ... +32767
- Yalnızca tam sayı: 7 / 2 = 3, 7 % 2 = 1
- Toplama, çıkarma, çarpma, bölme, mod

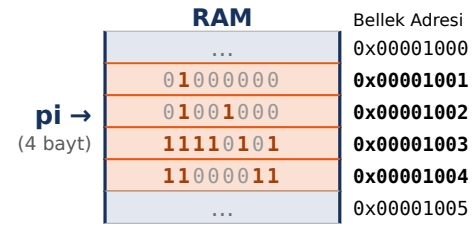
```
int puan = 50;
```



float — 4 bayt

- Virgüllü sayılar: 3.14 (kodda nokta ile)
- int'in 2 katı yer; 7 / 2 = 3.5 (küsuratlı)
- Çok geniş aralık, IEEE 754 biçimi

```
float pi = 3.14;
```



Karakterden Metne

Tek bir **char** 1 bayt yer kaplar. Birden fazla karakterden oluşan bir metin için her harfe RAM'de art arda (bitişik) 1 baytlık kutular ayrılır.

Toplam bellek alanı = karakter sayısı × 1 bayt

"Ada Lovelace" → 12 karakter (boşluk dahil) = **12 bayt**. Boşluk karakteri (SP, ASCII 32) de 1 bayt yer kaplar.

Neden önemli? Büyük veri türleri hem RAM'de daha fazla yer kaplar hem de işlemciyi daha fazla meşgul eder; bu yüzden gerekmedikçe **int** yerine **float** kullanılmamalıdır.

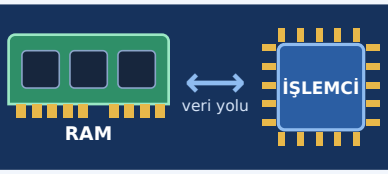
RAM
karakter · 8 bit · ASCII

			Bellek Adresi
A	01000001	65	0x00001000
d	01100100	100	0x00001001
a	01100001	97	0x00001002
SP	00100000	32	0x00001003
L	01001100	76	0x00001004
o	01101111	111	0x00001005
v	01110110	118	0x00001006
e	01100101	101	0x00001007
l	01101100	108	0x00001008
a	01100001	97	0x00001009
c	01100011	99	0x0000100A
e	01100101	101	0x0000100B

Not: Anlatımda C++ ve 16 bit sistemler (Arduino Uno, Turbo C) esas alınmıştır; böylece veri türlerinin donanımsal mantığı daha rahat anlaşılır. Derslerimizde Python kullanılacaktır; Python'da veri türleri ve bellek yapısı farklı işler.

RAM ve İşlemci İlişkisi

İşlemci, RAM'den aldığı veriyi işleyip tekrar RAM'e yazar.
Veri ne kadar büyükse bu yolculuk o kadar uzun sürer.



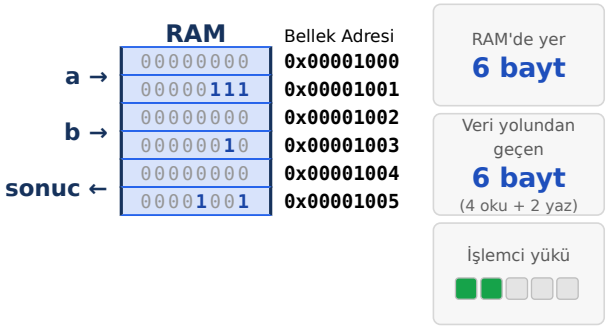
Büyük veri türü (daha çok bayt) → **RAM'de daha fazla yer** → **Veri yolunda daha çok bayt taşınır** → **İşlemci daha uzun süre meşgul olur**

İşlemci RAM ile nasıl çalışır? (sonuc = a + b)

1. **Oku:** işlemci **a** ve **b** değerlerini RAM'den veri yolu ile alır.
2. **İşle:** toplama işlemini yapar.
3. **Yaz:** sonucu RAM'e geri yazar.

int — 2 bayt

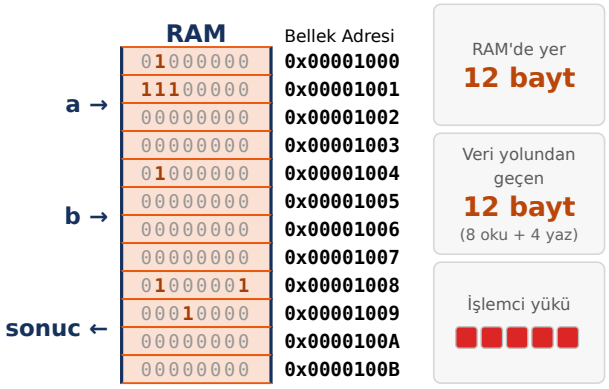
```
int sonuc = a + b;
```



a = 7, b = 2 için toplam sadece **6 bayt** RAM'e yazılıp okunur. Küçük bir işlemcide (ör. 8 bitlik Arduino Uno) bile 2 küçük adımda (düşük bayt, sonra yüksek bayt) toplanır.

float — 4 bayt

```
float sonuc = a + b;
```

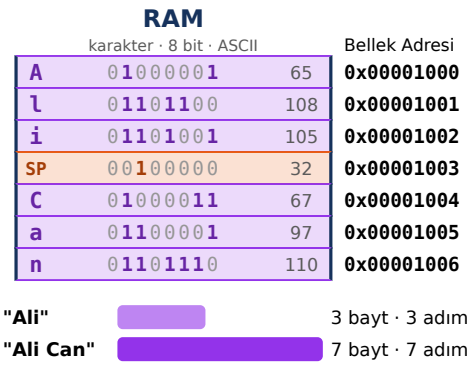


Aynı işlem için **12 bayt** taşınır: int'in **2 katı**. FPU olmayan işlemcilerde (ör. Arduino Uno) float, yazılımla çok daha fazla adımda hesaplanır.

Metin (char dizisi): her karakter 1 bayt, boşluk da dahil

```
char isim[7] = {'A', 'l', 'i', ' ', 'C', 'a', 'n'};
```

- **"Ali Can"** = 6 harf + 1 boşluk = **7 karakter = 7 bayt**
- **Boşluk da bir karakterdir:** ASCII 32, RAM'de 1 bayt yer kaplar ve işlemci onu da okur.
- Metni kopyalarken, karşılaştırırken ya da ekrana yazarken işlemci **her karakter için bir adım** atar.



1. **Tam sayı: int** — Küsurat gerekmedikçe float kullanma: daha çok RAM, daha çok işlemci süresi.
2. **Uygun türü seç** — Değerin aralığına yeten en küçük türü kullan.
3. **Metin uzadıkça** — Boşluklar dahil her karakter RAM'de yer tutar ve işlenir; yük artar.
4. **Eski donanım** — RAM ve işlemci sınırlıydı; veri türleri verimli bellek yönetimi için doğdu.

Not: Bayt sayıları C++ ve 16 bit sistemler (Arduino Uno, Turbo C) esas alınarak verilmiştir. Gerçek işlem süresi işlemciye göre değişir; ama RAM'de yer kaplayan her bayt, veri yolundan geçer ve işlemcinin işini artırır. Python'da bellek yapısı farklıdır.

12. Python Dili

Python; yüksek seviyeli, yorumlanan, nesne yönelimli ve geniş bir kütüphane ekosistemine sahip bir programlama dilidir. 1991 yılında Guido van Rossum tarafından geliştirilen bu dil, adını van Rossum'un beğeniyle takip ettiği İngiliz komedi grubu Monty Python'ın Monty Python's Flying Circus adlı televizyon programından almıştır.

Günümüzde bilgisayar donanımları, programlama dillerinin ilk ortaya çıktığı dönemlere kıyasla inanılmaz bir kapasiteye ulaşmıştır. Bu sayede Python gibi dillerin getirdiği çalışma zamanı maliyeti modern donanımlarla büyük ölçüde tolere edilmekte; çoğu uygulama için fazlasıyla yeterli ve pratik bir performans sunmaktadır.

"Notlarımızın bundan sonraki bölümlerinde vereceğimiz örnekler **Python programlama dili** için olacaktır."

C Dili

İki Sayının Toplamını Alan Kod Örneği

```c
#include <stdio.h>

int main() {
    int sayi1, sayi2, toplam;

    printf("Birinci sayi: ");
    scanf("%d", &sayi1);

    printf("Ikinci sayi: ");
    scanf("%d", &sayi2);

    toplam = sayi1 + sayi2;
    printf("Toplam: %d\n", toplam);

    return 0;
}
```

Python Dili

```python
sayi1 = int(input("Birinci sayı: "))
sayi2 = int(input("İkinci sayı: "))

toplam = sayi1 + sayi2
print(f"Toplam: {toplam}")
```

Sarı ve Mavi Yılanlar:
Python ismi her ne kadar Monty Python grubundan gelse de resmi logo, Antik Yunan mitolojisindeki **"Ouroboros"** (kendini besleyen/tamamlayan döngü) esintileri taşıyan iki stilize yılandan oluşur. Sarı ve mavi renkler; dilin esnekliğini, farklı sistemlerle kolay entegre olabilmesini ve geliştirici dostu yapısını temsil eder.

Neden Günümüzün En Popüler Dili?

- **Sözdizimi Sadeliği:** C ve Java gibi dillerdeki süslü parantez ({}) ve noktalı virgül (;) kalabalığı yerine girintilere (**indentation**) dayanır. Kod, neredeyse günlük İngilizce gibi okunur.
- **Geniş Ekosistem:** Yapay zekâdan veri bilimine, web geliştirmeden siber güvenliğe kadar her alanda devasa kütüphane desteği sunar.
- **Endüstri Standardı:** TIOBE ve GitHub Octoverse gibi küresel programlama dili endekslerinde son yıllarda sürekli 1. sırada yer almaktadır.

Yaygın Kullanım Alanları:

- **Yapay Zekâ & Makine Öğrenmesi:** (TensorFlow, PyTorch, Scikit-learn)
- **Veri Analizi & Görselleştirme:** (Pandas, NumPy, Matplotlib)
- **Web Geliştirme (Backend):** (Django, FastAPI, Flask)
- **Otomasyon & Scripting:** (Ağ yönetimi, veri kazıma, sistem görevleri)

13. Tür Dönüşümü (Type Conversion / Type Casting)

Bilgisayarlar bellekteki verileri türlerine göre farklı formatlarda saklandığını gördük. Bir verinin türünü başka bir türe çevirme işlemine **tür dönüşümü** denir.

13.1. Kullanıcı Girişlerinin Sayıya Dönüştürülmesi (Metin → Sayı)

Klavyeden girilen veya bir metin dosyasından okunan tüm veriler, sisteme ilk olarak karakter dizisi (metin / string) biçiminde ulaşır.

- Kullanıcı ekrana 25 yazdığında bilgisayar bunu sayısal 25 olarak değil; yan yana duran **'2'**(ASCII: 50) ve **'5'**(ASCII: 53) karakterleri olarak algılar.
- Karakter tabanlı bir veriyle matematiksel işlem yapılamaz. Örneğin iki metinsel veri toplanmak istendiğinde matematiksel toplama ($25 + 10 = 35$) yerine metin birleştirme ("25" + "10" = "2510") gerçekleşir veya derleyici hata verir.
- Bu değerlerin matematiksel işlemlere girebilmesi için metin biçiminden uygun sayı türüne (int veya float) dönüştürülmesi (parse/cast edilmesi) şarttır.

Örnek Python Kodu: Kullanıcının girdiği tam sayı değere 25 ekleyerek sonucu ekrana yazdıran kod.

```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Python Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```

```python
puan = int(input("Puan Giriniz: "))
```

Program, kullanıcıdan "Puan Giriniz:" iletisiyle bir tam sayı değeri beklemektedir. Python'da input() fonksiyonu klavyeden veri almak için kullanılır. Ancak daha önce belirttiğimiz gibi, input() fonksiyonu girilen veriyi ne olursa olsun her zaman bir metin (string) olarak okur. Bu metinsel değerin matematiksel hesaplamalarda veya karşılaştırmalarda kullanılabilmesi için int() fonksiyonu ile tam sayıya dönüştürülmesi şarttır.

```python
veri = input("Puan Giriniz: ")  # 1. Aşama: Metin olarak alındı ("50")
puan = int(veri)                # 2. Aşama: Tam sayıya dönüştürüldü (50)
```

Dileyenler bu işlemi tek satırda iç içe yapmak yerine, önce veriyi bir değişkene metin olarak alıp ardından ikinci bir satırda **int()** fonksiyonuyla sayıya dönüştürerek iki aşamada da gerçekleştirebilir. Fakat bu tek satır kod yazmaya göre daha uzundur.

```python
# Kullanıcı girişleri metin olarak alınır
birinci_sayi = input("Birinci Sayıyı Girin: ")
ikinci_sayi = input("İkinci Sayıyı Girin: ")
ucuncu_sayi = input("Üçüncü Sayıyı Girin: ")

# Girilen 3 değişken float türüne dönüştürülür
birinci_sayi = float(birinci_sayi)
ikinci_sayi = float(ikinci_sayi)
ucuncu_sayi = float(ucuncu_sayi)

bolum = (birinci_sayi + ikinci_sayi) / ucuncu_sayi
print("Bölüm:", bolum)
```

Örnek Python Kodu : Kullanıcı girişlerinin **metin türünden float** tipine dönüşümü yapılır.

Python Ekran Çıktısı

```
Birinci Sayıyı Girin: 1.2
İkinci Sayıyı Girin: 1.8
Üçüncü Sayıyı Girin: 2.5
Bölüm: 1.2
```

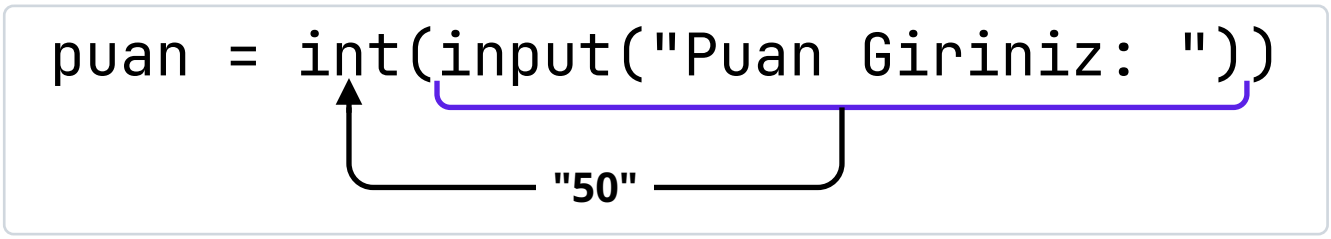
13.1.1. `int(input())` Nasıl Çalışır?

```
puan = int(input("Puan Giriniz: "))
```

Bu satır **içten dışa** doğru çalışır: önce en içteki `input()`, sonra onu saran `int()`, en son da atama (**=**).

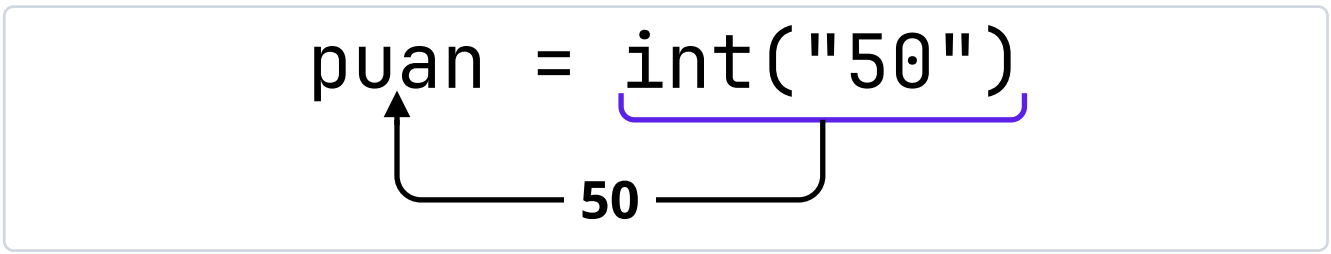
Adım 1: `input()` çalışır ve `int()` fonksiyonuna gider

Kullanıcı klavyeden `50` yazıp Enter'a basar. `input()` fonksiyonu bu değeri her zaman **metin** olarak döndürür; yani elimize geçen şey sayı değil, tırnak içinde `"50"` olur. Bu değer, dıştaki `int()` fonksiyonuna gider.



Adım 2: `int()` çalışır ve sonuç `puan` değişkenine gider

`input()` çağrısı yerini döndürdüğü değere bıraktı; satır artık şu hale geldi: `puan = int("50")`.
`int()` fonksiyonu `"50"` metnini tam sayıya çevirir ve sonuç olan `50` değeri `puan` değişkenine atanır.



Tam Sayı Girişi Örneği

```
1 puan = int(input("Puan Giriniz: "))
2 print(puan)
3 print(type(puan))
```

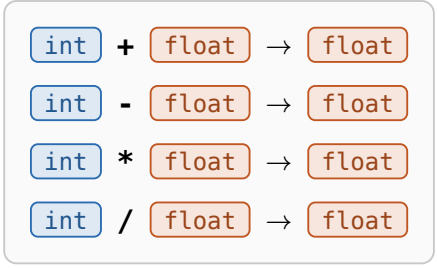
Ekran Çıktısı

```
Puan Giriniz: 50
50
<class 'int'>
```

> **Dikkat:** `int()` olmasaydı `puan` değişkeninde `"50"` yani **metin** olurdu ve `puan + 10` gibi bir işlem hata verirdi. Sayıyla işlem yapacaksak `int()` (ya da ondalıklı değerler için `float()`) ile dönüştürmemiz gerekir.

13.2. Otomatik (Örtük / Implicit) Dönüşüm:

Farklı türler aynı matematiksel işleme girdiğinde, veri kaybını önlemek için küçük tür otomatik olarak daha geniş ve kapsayıcı olan türe yükseltilir:




```
1 adet = 5           # int türünde
2 birim_fiyat = 2.5   # float türünde
3
4 # Python arka planda 'adet' değerini 5.0 yapar
5 toplam_tutar = adet * birim_fiyat # 5.0 * 2.5 = 12.5(float)
6
7 print(toplam_tutar) # Çıktı: 12.5
```

13.3. Bilinçli (Açık / Explicit) Dönüşüm:

Programcının veri kaybını göze alarak veya işlemi yönlendirmek için türü zorla değiştirmesidir.

Hassasiyeti Korumak İçin Tam Sayıyı Kesirli Sayıya Çevirmek (int ⟶ float): Daha hassas değerler gerektiğinde tam sayı değerleri kasten kesirli değerlere dönüştürülebilir.

```python
puan = 100

ondalikli_puan = float(puan)  # 100 -> 100.0

print(ondalikli_puan)  # Çıktı: 100.0
```

Ondalık Kısmı Atmak İçin (float ⟶ int): Ondalıklı bir sayının virgülden sonraki kısmını silip yalnızca tam kısmını elde etmek gerektiğinde bu dönüşümden yararlanılır.

```python
ortalama = 85.85
# Virgülden sonrası silinir
tam_not = int(ortalama) #tam_not = 85 olur

print(tam_not)  # Çıktı: 85
```

14. Değişken Tanımlama Kuralları

Yazılan kaynak kodların Python yorumlayıcısı tarafından doğru şekilde işlenebilmesi için değişken adları belirlenirken Python'un söz dizimi (**syntax**) kurallarına uyulması zorunludur.

- **Karakter Kısıtlaması:** Değişken isimleri sadece alfabetik karakterler (**A-Z, a-z**), rakamlar (**0-9**) ve alt çizgi (_) karakterini barındırabilir. Bunların dışındaki hiçbir özel sembol (nokta, virgül, soru işareti, tire, boşluk vb.) değişken isminde yer alamaz.
- **Başlangıç Kriteri:** Bir değişken ismi kesinlikle bir rakam ile başlayamaz. İsimler ya alfabetik bir karakterle ya da alt çizgi (_) işaretiyle başlamak zorundadır. Örneğin 3etap_skor hatalı bir söz dizimi iken, **etap_skor_3** veya **_etap3** kurallara uygundur.
- **Büyük-Küçük Harf Duyarlılığı (Case-Sensitivity):** Python dili, büyük ve küçük harfleri farklı karakterler olarak değerlendirir. Bu nedenle aynı harflerden oluşsa bile yazımında büyük-küçük harf farkı bulunan kelimeler, bellekte tutulan verilere erişmek için kullanılan farklı değişken adları olarak kabul edilir. Örneğin **asistan_adi**, **Asistan_Adi** ve **ASISTAN_ADI** ifadeleri Python yorumlayıcısı tarafından birbirinden bağımsız üç farklı değişken adı olarak değerlendirilir.
- **Resmî Dil Anahtarları (Reserved Words):** Python dilinin kendi iç mekanizmasını yönetmek ve komutları işletmek adına kendine ayırdığı bazı özel kelimeler (keywords) değişken adı olarak kullanılamaz. Örneğin **if**, **else**, **True**, **False**, **while**, **for** gibi kelimeler Python için özel bir anlam ifade ettiğinden doğrudan değişken ismi yapılamaz.
- **Türkçe Karakter Kullanımı:** Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken **Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!**.

Türkçe Karakterlerin İngilizce Karşılıkları

Türkçe	Dönüşüm	İngilizce
ç / Ç	⟶	**c / C**
ğ / Ğ	⟶	**g / G**
ı / İ	⟶	**i / I**
ö / Ö	⟶	**o / O**
ş / Ş	⟶	**s / S**
ü / Ü	⟶	**u / U**

Değişken Kullanım Örnekleri

Hatalı Kullanım	Dönüşüm	Doğru Kullanım
doğumTarihi	⟶	**dogumTarihi**
ürünFiyatı	⟶	**urunFiyati**
şubeKodu	⟶	**subeKodu**
kayıtDurumu	⟶	**kayitDurumu**
şifresi	⟶	**sifresi**
öğrenci_yaşı	⟶	**ogrenci_yasi**
ücret_miktarı	⟶	**ucret_miktari**
ilk_değer	⟶	**ilk_deger**

15. Değişken Adlandırma Standartları (Notasyonlar)

15.1. Yılan Gösterimi (snake_case)

Tüm harflerin küçük yazıldığı ve kelimelerin birbirine alt çizgi (_) karakteriyle bağlandığı yöntemdir. Python literatüründe değişken ve fonksiyon isimleri için bu standardın kullanılması (örneğin **kullanici_giris_tarihi, asistan_bellek_kapasitesi** vb.) kurumsal olarak tavsiye edilir.

15.2. Camel Case (Deve Notasyonu) Kuralı

Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

Değişken Adlandırma Kuralları (Naming Conventions)

Yanlış / Zor Okunan Biçim	snake_case (Python)	camelCase
personelsayisi	personel_sayisi	personelSayisi
dogumyil	dogum_yil	dogumYil
ogrencinotortalamasi	ogrenci_not_ortalamasi	ogrenciNotOrtalamasi
toplamfiyat	toplam_fiyat	toplamFiyat
kullaniciadi	kullanici_adi	kullaniciAdi
maksimumhizsiniri	maksimum_hiz_siniri	maksimumHizSiniri
sonodemetarihi	son_odeme_tarihi	sonOdemeTarihi

16. Sabitler

Bir yazılım mimarisinde programın çalışma süresince değiştirilmemesi amaçlanan ve aynı değeri koruması beklenen verilere sabit (**constant**) adı verilir. Sabit olarak kullanılması amaçlanan isimler, tüm harfleri büyük olacak ve kelimeler alt çizgiyle (_) ayrılacak biçimde yazılır.

```
# Doğru Sabit Kullanım Standardı (BÜYÜK HARF)
PI_SAYISI = 3.14159
MAKSIMUM_DENEME_SAYISI = 3
SISTEM_PORT_NUMARASI = 443
VARSAYILAN_DIL = "TR"
KARSILAMA_MESAJI = " Sisteme Hoş Geldiniz!"
GELISTIRICI_MODU_ACIK = True
```

17. Açıklama Satırı (Comment line)

Yazılım projelerinde, derleyicinin görmezden geldiği ancak yazılımcıların kendileri için veya diğer yazılımcılar için bıraktığı notlara açıklama satarı (comment line) denir. Python'da iki tür açıklama satırı uygulaması vardır.

- **#** ile başlayan tek satırlık açıklama satırı
- **"""..."""** Çok satırlı açıklama satırları üç adet tırnak ile açılan ve yine üç adet tırnak ile kapanan alan içine yazılan açıklama ifadeleri ile yapılır.

Açıklamasız

```python
a = 50
b = 20
c = a * b / 100
print(c)
```

Açıklamalı

```python
# Ürünün fiyatı
a = 50
# İndirim oranı (%)
b = 20
# İndirim miktarı
c = a * b / 100
print(c)
```

Açıklama Satırının Kullanım Amaçları

```python
# 1) Kodu açıklamak için
yas = 17  # öğrencinin yaşı

# 2) Kodu geçici olarak devre dışı bırakmak için
print("Merhaba")
# print("Bu satır çalışmaz")


# 3) Yapılacakları not etmek için
# TODO: Kullanıcıdan yaş bilgisi alınacak
```

Çok Satırlı Açıklama ile Program Başlığı

```python
"""
Program : Daire Alanı Hesaplama
Yazar   : Ahmet Tuna POTUR
Tarih   : 01.11.2026
Açıklama: Yarıçapı verilen dairenin alanını hesaplar.
"""
pi = 3.14        # pi sayısı (yaklaşık değer)
r = 5            # yarıçap
alan = pi * r * r
print(alan)      # Ekrana 78.5 yazar
```

> **Dikkat:** Açıklama satırları programın çalışmasını etkilemez, ancak kodun okunabilirliğini artırır. Gereksiz ve her satır için tekrarlanan açıklamalardan kaçınılmalı; kodun **ne yaptığından çok neden yaptığı** anlatılmalıdır.

```python
# Bu tam satırlık açıklama
x = 10  # satır içi (inline) açıklama
```

```python
# Bu fonksiyon iki sayıyı toplar,
# ve geriye döndürür.
def topla(a, b):
    return a + b
# fonksiyon sonlandırıldı
```

```python
"""
Bu teknik olarak bir string değişmezidir, ancak bir
değişkene atanmadığı için yorum bloğu gibi davranır.
"""
print("Kod çalışıyor")
```

18. Operatörler ve İşlem Önceliği

Operatör, bir veya birden fazla değer üzerinde **matematiksel hesaplama** yapan, değerleri **karşılaştıran** veya **mantıksal işlemler** yürüterek yeni bir sonuç üreten özel **sembol** ya da **anahtar kelimelerdir**. Operatörün üzerinde işlem yaptığı değerlere veya değişkenlere **operand** (işlenen) denir.

```
sonuc = 5 + 3
```

- **5** ve **3** → **operand** (işlenen değerler)
- **+** → **operatör** (işlemi yapan sembol)
- **=** → **atama operatörü** (sonucu `sonuc` değişkenine yerleştirir)

Python'daki operatörleri dört ana grupta inceleyeceğiz

Grup	Ne işe yarar?	Operatörler
Atama	Bir değişkene değer yerleştirir.	`= += -= *= /= //= %= **=`
Aritmetik	Matematiksel hesap yapar.	`+ - * / // % **`
Karşılaştırma	İki değeri kıyaslar; sonuç `True` veya `False` olur.	`== != > < >= <=`
Mantıksal	Koşulları birleştirir veya tersine çevirir.	`and or not`

18.1. Atama Operatörleri

Matematikte "eşittir" anlamına gelen **=** sembolü, programlama dillerinde atama operatörüdür; sağındaki değeri alır ve soldaki değişkene aktarır.

> **Atama Her Zaman Sağdan Sola Çalışır** Eğer bir hesaplama işlemi yapılıyorsa, önce operatörün sağ tarafı hesaplanır; ardından elde edilen sonuç soldaki değişkene atanır.

18.1.1. Atama Operatörü (=): Değişken Oluşur veya Güncellenir

Bir değişkene ilk kez değer atadığımızda, atanan değer için bellek (RAM) üzerinde bir yer ayrılır ve değer bu alana yazılır. Değişken adı ise bu bellek alanını gösteren bir **etikettir (işaretçidir)**.

```
yas = 15          # yas değişkeni oluştu, bellekte 15 değeri saklandı
isim = "Ayşe"     # isim değişkeni oluştu, bellekte "Ayşe" saklandı
pi_sayisi = 3.14  # pi_sayisi değişkeni oluştu
```

Var olan bir değişkene yeniden atama yaptığımızda yeni bir değişken oluşmaz; değişkenin bulunduğu bellek alanında ki eski değerin üzerine yeni değer yazılır ve eski değer kaybolur.

```
puan = 50
print("Puan:", puan)
puan = 75
print("Güncel puan:", puan)
```

Ekran Çıktısı

```
Puan: 50
Güncel puan: 75
```

Güncelleme yaparken sağ tarafta değişkenin önceki değeri ile hesaplama yapılır, sonra sonuç değişkene atanır.

```
puan = 50
puan = puan + 25 # 50 + 25 = 75 hesaplanır, puan'a atanır
print(sayac)  # 75
```

Python'da tür bilgisi değere bağlı olduğu için, değişken yeni atanan değerin türünü alır:

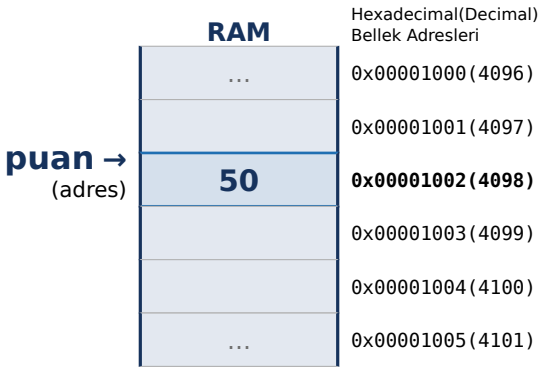
```
puan = 50     # int
puan = 55.43  # artık float
puan = "elli" # artık str
```

Bellekte Değerin Güncellenmesi

1 Değer Atama

Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

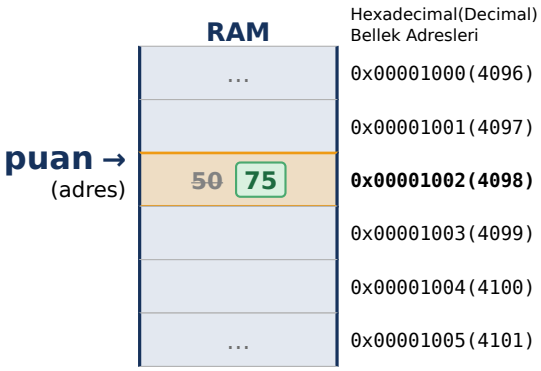
```
puan = 50
```



2 Değer Güncelleme

Giriş yapılan **75** değeri, RAM'de bulunan **puan** değişkeninin önceki değerinin üzerine yazılır.

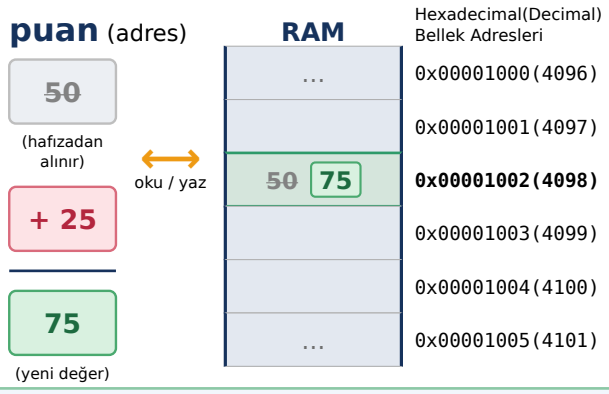
```
puan = 75
```



3 Mevcut ile Güncelleme

Güncellenen değişkenin eski değeri kullanılarak yeni değer yazılır.

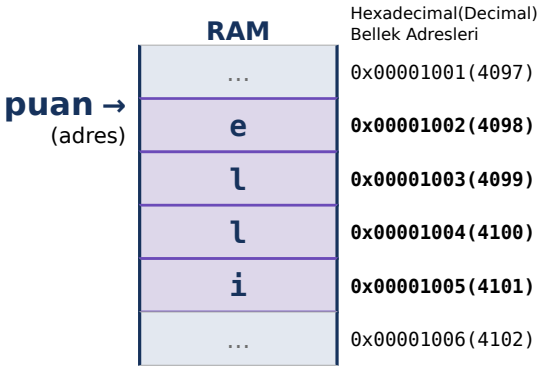
```
puan = puan + 25
```



4 Değişken Türü ile Güncelleme

RAM'deki sayısal alan silinir, yerine metin dizisi oluşturulur.

```
puan = "elli"
```



18.1.2. Bileşik Atama Operatörleri

sayac = sayac + 1 gibi **"değişkeni kendi değeriyle işleme sokup tekrar kendisine yazma"** işlemi o kadar sık yapılır ki bunun için kısaltmalar vardır:

Bileşik Atama Operatörleri

Operatör	Kısa Yazım	Uzun Karşılığı	`a = 10` iken sonuç
`+=`	`a += 3`	`a = a + 3`	13
`-=`	`a -= 3`	`a = a - 3`	7
`*=`	`a *= 3`	`a = a * 3`	30
`/=`	`a /= 4`	`a = a / 4`	2.5
`//=`	`a //= 4`	`a = a // 4`	2
`%=`	`a %= 4`	`a = a % 4`	2
`**=`	`a **= 2`	`a = a ** 2`	100

```python
bakiye = 100
bakiye += 50
bakiye -= 30
bakiye *= 2
print("Bakiye:", bakiye)

mesaj = "Merhaba"
mesaj += " Dünya"
print(mesaj)
```

Bileşik Atama Operatörleri Örnekleri

Ekran Çıktısı

```
Bakiye: 240
Merhaba Dünya
```

18.2. Aritmetik Operatörler

Python Aritmetik Operatörleri: Matematiksel İşlemleri Yapar

Operatör	Adı	Örnek	Sonuç	Açıklama
+	Toplama	`10 + 3`	`13`	İki sayıyı toplar.
-	Çıkarma	`10 - 3`	`7`	Soldakinden sağdakini çıkarır.
*****	Çarpma	`10 * 3`	`30`	İki sayıyı çarpar.
/	Bölme	`10 / 4`	`2.5`	Sonuç her zaman `float` olur.
//	Tam bölme	`10 // 4`	`2`	Bölümün tam sayı kısmını verir (küsuratı atar).
%	Mod (kalan)	`10 % 4`	`2`	Bölmeden kalanı verir.
******	Üs alma	`2 ** 3`	`8`	Soldaki sayının sağdaki kuvvetini alır.

```python
a = 17
b = 5

print("Toplam:", a + b)
print("Fark:", a - b)
print("Çarpım:", a * b)
print("Bölüm:", a / b)
print("Tam bölüm:", a // b)
print("Kalan:", a % b)
```

Aritmetik operatörlerin sonuçları

Ekran Çıktısı

```
Toplam: 22
Fark: 12
Çarpım: 85
Bölüm: 3.4
Tam bölüm: 3
Kalan: 2
```

Not: Tam bölünen sayılarda bile sonuç `float` türüne dönüşür

```python
print(10 / 4)          # 2.5
print(10 / 5)          # 2.0   (2 değil!)
```

Tam Bölme (`//`) ve Mod (`%`): Bir bölme işlemini ilkokuldaki gibi düşünün: **bölünen = bölen × bölüm + kalan**. `//` operatörü bölümü, `%` operatörü kalanı verir.

```python
print(17 // 5)   # 3   (17 içinde 3 tane 5 var)
print(17 % 5)    # 2   (3 × 5 = 15, geriye 2 kalır)
```

Tam Bölme ve Mod Örnekleri

Bölünen ve Bölen	`//` (bölüm)	`%` (kalan)	Kontrol
17 ve 5	3	2	`3 * 5 + 2 = 17`
20 ve 4	5	0	`5 * 4 + 0 = 20`
100 ve 30	3	10	`3 * 30 + 10 = 100`

```python
# 1) Tek mi çift mi?
sayi = 7
print(sayi % 2)    # 1  -> kalan 1 ise sayı tektir
print(10 % 2)      # 0  -> kalan 0 ise sayı çifttir

# 2) Bir sayı diğerine tam bölünüyor mu?
print(15 % 5)      # 0  -> 15, 5'e tam bölünür
print(15 % 4)      # 3  -> tam bölünmez
```

Dakikayı saat ve dakikayı çevirme

```python
toplam_dakika = 135
saat = toplam_dakika // 60
dakika = toplam_dakika % 60
print(saat, "saat", dakika, "dakika")
```

Ekran Çıktısı

```
2 saat 15 dakika
```

Üs Alma (**)

```python
print(2 ** 3)     # 8       (2 × 2 × 2)
print(5 ** 2)     # 25
print(9 ** 0.5)   # 3.0     (karekök almak: 0.5 kuvveti)
```

Metinlerle `+` ve `*`:

+ operatörü metinleri birleştirir, ***** operatörü bir metni belirtilen sayı kadar tekrarlar

```python
ad = "Ali"
soyad = "Yılmaz"
print(ad + " " + soyad)
print("Ha" * 3)
print("-" * 20)
print("Yaş: " + str(15))
```

Ekran Çıktısı

```
Ali Yılmaz
HaHaHa
#("-" * 20)
Yaş: 15
```

18.3. Aritmetik Operatörlerde İşlem Önceliği

Bir ifadede birden fazla operatör varsa Python işlemleri işlem önceliğine göre matematikteki sırayla yapar. Önceliği yüksek olan işlem önce hesaplanır.

Aritmetik Operatörlerin İşlem Önceliği

Öncelik	Operatör	Açıklama
1 (en yüksek)	`( )`	Parantez içi her zaman önce hesaplanır.
2	`**`	Üs alma
3	`-x`	Tek operandlı eksi (negatif işareti)
4	`* / // %`	Çarpma, bölme, tam bölme, mod (kendi aralarında eşit öncelikli)
5 (en düşük)	`+ -`	Toplama, çıkarma (kendi aralarında eşit öncelikli)

Eşit öncelikli operatörler soldan sağa işlenir:

```python
print(10 - 4 - 3)      # 3      ((10 - 4) - 3)
print(100 / 10 * 5)    # 50.0   ((100 / 10) * 5)
```

```python
print(2 + 3 * 4)        # 14
# 1) 3 * 4 = 12
# 2) 2 + 12 = 14
```

```python
print((2 + 3) * 4)      # 20
# 1) (2 + 3) = 5
# 2) 5 * 4 = 20
```

```python
print(2 + 3 ** 2 * 2)    # 20
# 1) 3 ** 2 = 9
# 2) 9 * 2 = 18
# 3) 2 + 18 = 20
```

```python
print(7 + 9 / 3 * 2)    # 13.0
# 1) 9 / 3 = 3.0
# 2) 3.0 * 2 = 6.0
# 3) 7 + 6.0 = 13.0
```

İstisna: Üs alma (**), eşit öncelikli işlemlerin aksine **sağdan sola** çalışır.
Eksi işareti de ** operatöründen daha düşük önceliklidir:

```python
print(2 ** 3 ** 2)     # 512  -> 2 ** (3 ** 2) = 2 ** 9
print((2 ** 3) ** 2)   # 64
print(-2 ** 2)         # -4   -> -(2 ** 2)
print((-2) ** 2)       # 4
```

Pratik öneri: Sıradan emin değilseniz **parantez kullanın**. Parantez hem hatayı önler hem de kodu okuyan kişiye niyetinizi gösterir.

```python
n1 = 80
n2 = 90
n3 = 70
ortalama = (n1 + n2 + n3) / 3
print("Ortalama:", ortalama)

c = 25
f = c * 9 / 5 + 32
print("Fahrenheit:", f)
```

Not ortalaması ve sıcaklık dönüşümü

Ekran Çıktısı

```
Ortalama: 80.0
Fahrenheit: 77.0
```

```python
# Dikdörtgenin alanı ve çevresi
en = 6
boy = 4
alan = en * boy          # 24
cevre = 2 * (en + boy)   # 20
```

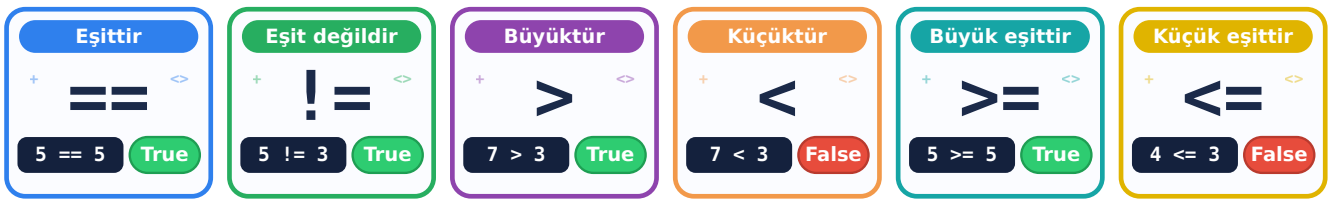
```python
# Bir sayının son basamağı ve
# son basamağı atılmış hali
sayi = 4827
print(sayi % 10)    # 7
print(sayi // 10)   # 482
```

18.4. Karşılaştırma Operatörleri

Karşılaştırma operatörleri iki değeri kıyaslar ve sonuç olarak her zaman `True` (doğru) veya `False` (yanlış) üretir. Bu iki değer `bool` (mantıksal) veri türündedir.

Python Karşılaştırma Operatörleri

Operatör	Anlamı	Örnek	Sonuç	Açıklama
==	Eşittir	`5 == 5`	`True`	İki değer eşitse `True` olur.
!=	Eşit değildir	`5 != 3`	`True`	İki değer farklıysa `True` olur.
>	Büyüktür	`7 > 3`	`True`	Soldaki, sağdakinden büyükse `True` olur.
<	Küçüktür	`7 < 3`	`False`	Soldaki, sağdakinden küçükse `True` olur.
>=	Büyük eşittir	`5 >= 5`	`True`	Soldaki büyük **veya** eşitse `True` olur.
<=	Küçük eşittir	`4 <= 3`	`False`	Soldaki küçük **veya** eşitse `True` olur.



Bir sayının çeşitli karşılaştırma sonuçları

```
sayi = 15
print(sayi > 10)
print(sayi == 15)
print(sayi != 15)
print(sayi <= 14)
print(sayi % 2 == 0)
```

Ekran Çıktısı

```
True
True
False
False
False
```

Not: Karşılaştırmanın sonucu da bir değerdir; bir değişkende saklanabilir:

```
yas = 16
resit_mi = yas >= 18
print(resit_mi)   # False
```

Dikkat: = ve == aynı şey değildir.

```
x = 5 # ATAMA: x'e 5 değerini yerleştir
x == 5  # KARŞILAŞTIRMA: x, 5'e eşit mi? -> True
```

Metinleri Karşılaştırma Metinler de karşılaştırılabilir. == operatörü büyük/küçük harfe duyarlıdır:

```
print("elma" == "elma")    # True
print("elma" == "Elma")    # False  (büyük/küçük harf farklı)
print("elma" != "armut")   # True
print("a" < "b")           # True   (alfabetik sıra / karakter kodu)
```

Farklı Türlerin Karşılaştırılması

```
print(3 == 3.0)        # True   (int ile float değer olarak eşit)
print("5" == 5)        # False  (metin ile sayı aynı şey değildir)
print(int("5") == 5)   # True   (türü dönüştürünce eşit)
```

Ondalıklı sayılarda == kullanırken dikkat: Bilgisayar ondalıklı sayıları tam kesinlikle saklayamaz. Bu yüzden `float` değerlerin == ile karşılaştırılması güvenilir değildir.

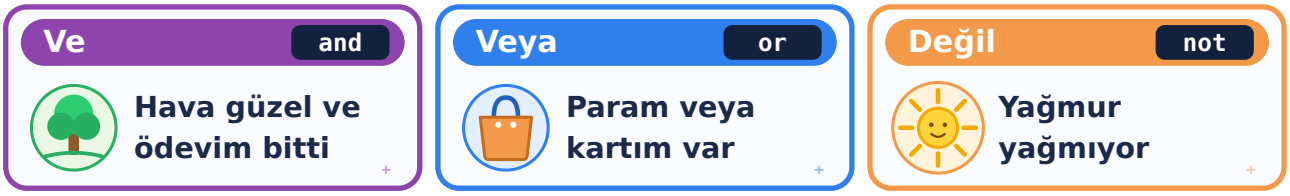
```
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False
```

18.5. Mantıksal Operatörler

Mantıksal operatörler, `True` / `False` değerlerini ve karşılaştırma sonuçlarını birleştirmek veya tersine çevirmek için kullanılır. Üç tanedir: `and` (ve), `or` (veya), `not` (değil).

Python Mantıksal Operatörleri

Operatör	Anlamı	Ne zaman True olur?	Günlük Hayattan Örnek
and	Ve	İki koşul da `True` ise.	"Hava güzel **ve** ödevim bitti ise parka giderim."
or	Veya	En az bir koşul `True` ise.	"Param **veya** kartım varsa alışveriş yapabilirim."
not	Değil	Koşul `False` ise (tersine çevirir).	"Yağmur yağmıyorsa dışarı çıkarım."



Doğruluk Tabloları

and: Hepsi doğru olmalı

A	B	A and B
True	True	**True**
True	False	False
False	True	False
False	False	False

or: Biri doğru olsa yeter

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	**False**

not:Tersine çevirir

A	not A
True	False
False	True

Örnek Kod

```
print(True and True)  #True
print(True and False) #False
print(False or True) #True
print(False or False) #False
print(not True) #False
print(not False) #True
```

```
yas = 20
ehliyet = True

print(yas >= 18 and ehliyet)
print(yas < 18 or yas > 60)
print(not ehliyet)
```

Karşılaştırma operatörleriyle birlikte kullanım

Ekran Çıktısı

```
True
False
False
```

Örnek: Aritmetik ve Mantıksal Koşulların Birlikte Kullanımı

```
sayi = 15
print(sayi % 3 == 0 and sayi % 5 == 0)   # True  (hem 3'e hem 5'e bölünüyor)
print(sayi % 2 == 0 or sayi > 10)        # True  (çift değil ama 10'dan büyük)
print(not (sayi > 20))                   # True
```

Örnek: Adım adım mantıksal ifade

```
not_degeri = 65
devam = 20

gecti = not_degeri >= 50 and devam <= 30
# 1) not_degeri >= 50  ->  65 >= 50  ->  True
# 2) devam <= 30       ->  20 <= 30  ->  True
# 3) True and True     ->  True
print(gecti)   # True
```

Kısa Devre (Short-Circuit): Python mantıksal ifadeyi **gereksiz yere sonuna kadar hesaplamaz**:

- **and** ifadesinde soldaki `False` ise sonuç kesindir (`False`); sağ taraf hiç değerlendirilmez.
- **or** ifadesinde soldaki `True` ise sonuç kesindir (`True`); sağ taraf hiç değerlendirilmez.

Bu özellik, hataya yol açabilecek bir işlemi önce kontrol etmek için çok kullanışlıdır.

```
  x = 0
sayi = 12
metin = ""

# 1. Sıfıra bölme koruması: x != 0 False olduğu için sağdaki 20/x hiç çalışmaz (Hata vermez)
print(x != 0 and 20 / x > 2)          # False
# 2. or koruması: x == 0 True olduğu için sağdaki 100/x çalıştırılmaz
print(x == 0 or 100 / x == 5)         # True
# 3. Boş metin kontrolü: len(metin) > 0 False olduğu için metin[0] index hatası vermez
print(len(metin) > 0 and metin[0] == "A")  # False
# 4. Sol taraf True olunca and devam eder: Bu kez sağ taraf çalışır ve hesaplanır
print(sayi > 10 and sayi % 3 == 0)    # True
# 5. or ilk doğruyu bulunca durur: sayi > 5 True olduğundan sağdaki ağır koşula bakılmaz
print(sayi > 5 or sayi / x == 1)      # True
# 6. and zinciri: Sol taraf False olursa sonraki hiçbir işlem çalışmaz
print(x > 0 and sayi > 10 and 50 / x > 1)  # False
```

18.6. Operatörler Arası İşlem Önceliği

Bir ifadede aritmetik, karşılaştırma ve mantıksal operatörler birlikte bulunabilir. Python bunları şu sırayla işler: **önce aritmetik, sonra karşılaştırma, en son mantıksal**.

Operatörler Arası İşlem Önceliği

Sıra	Grup	Operatörler	Not
1 (en yüksek)	Parantez	`( )`	Her zaman en başta işlenir.
2	Aritmetik	`**`	Sağdan sola işlenir.
3	Aritmetik	`-x`	Tek operandlı eksi.
4	Aritmetik	`* / // %`	Soldan sağa işlenir.
5	Aritmetik	`+ -`	Soldan sağa işlenir.
6	Karşılaştırma	`== != > < >= <=`	Hepsi eşit öncelikli.
7	Mantıksal	`not`	Mantıksal operatörler içinde en yüksek öncelikli.
8	Mantıksal	`and`	`not` operatöründen sonra işlenir.
9 (en düşük)	Mantıksal	`or`	En son işlenir.
En sonda	Atama	`= += -= ...`	Sağ taraf tamamen hesaplanır, sonra atanır.

Gerçek hayat, kampa katılım kuralı

```
yas = 15
veli_izni = True
saglik_raporu = False

katilabilir = yas >= 14 and yas <= 17 and (veli_izni or saglik_raporu)
# 1) yas >= 14                      ->  True
# 2) yas <= 17                      ->  True
# 3) (veli_izni or saglik_raporu)   ->  True or False  ->  True
# 4) True and True and True         ->  True
print(katilabilir)  #True
```

Özet:

- Operatör, değerler üzerinde işlem yapan semboldür.
- **=** bir **atama** operatörüdür: ilk atamada değişken oluşur, sonraki atamalarda değeri güncellenir.
- Aritmetik operatörler sayısal sonuç, karşılaştırma ve mantıksal operatörler `bool` sonuç üretir.
- Öncelik sırası: **parantez** → **aritmetik** → **karşılaştırma** → **`not`** → **`and`** → **`or`**.
- Emin olmadığınız yerde parantez kullanın.

19. Fonksiyonların Kullanımı

“Kod tekrarı; aynı kodun farklı yerlerde yeniden yazılmasıdır. Hata vermez fakat kodun bakımını ve okunmasını zorlaştırır. Bu durumdan kaçınma ilkesine DRY (Don’t Repeat Yourself - Kendini Tekrar Etme) denir.”

Fonksiyonlar kod tekrarını önleyen en temel araçlardandır. Belirli bir görevi yerine getiren kod blokları bir fonksiyon çatısı altında toplanır ve ihtiyaç duyuldukça yalnızca fonksiyonun adı çağrılır. Fonksiyonlar, kendilerine gönderilen argümanları (girdi değerlerini) işleyerek ekrana bir çıktı verebilir veya hesaplanan sonucu geriye bir değer olarak döndürebilir.

Aşağıda, kesirli bir sayının virgülden sonraki iki basamağını elde etmek için kullanılan manuel bir mantık gösterilmiştir. Bu işlem için her defasında 5 satır kod yazdığımızı varsayalım. Programın farklı yerlerinde bu işlemi yalnızca üç kez tekrar etmemiz bile 15 satırlık gereksiz bir kod kalabalığı yaratacaktır. Oysa bunun yerine Python’ın hazır `round()` fonksiyonunu kullandığımızda aynı işlem tek bir satıra iner.

Fonksiyonları Kullanmanın Yarattığı Fark

```python
sayi = 3.14159
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #3.14
```

↓

```python
round(3.14159, 2) #3.14
```

```python
sayi = 5.03123
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #5.03
```

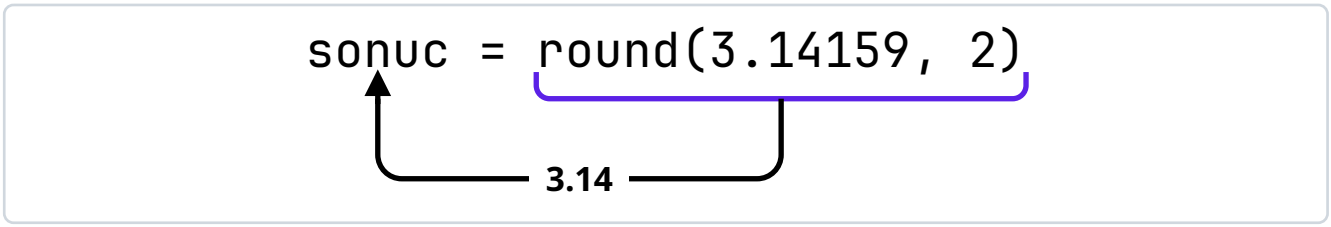
↓

```python
round(5.03123, 2) #5.03
```

```python
sayi = 7.21013
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #7.21
```

↓

```python
round(7.21013, 2) #7.21
```



Yukarıdaki örnekte **`round()`** fonksiyonunun kullanımı gösterilmektedir. Bir fonksiyonu çalıştırmak (**çağırmak**) için fonksiyonun adından hemen sonra parantez açılıp kapatılır: **`round()`**.

`round()` fonksiyonu dışarıdan girdi kabul eden bir fonksiyondur. Fonksiyon çağrılırken parantez içerisine yazılan bu değerlere **argüman** denir. Burada girilen: **Birinci değer (`3.14159`):** İşlem yapılacak olan ondalıklı sayıyı, **İkinci değer (`2`):** Bu sayının virgülden sonra kaç basamağa yuvarlanacağını belirtir.

Fonksiyonun kendi içinde hesapladığı sonucu çağrıldığı yere iletmesine **değer döndürme (return)** denir. Yukarıdaki örnekte **`round()`** fonksiyonunun döndürdüğü sonuç, **=** atama operatörü kullanılarak **`sonuc`** isimli değişkene aktarılmıştır.

Fonksiyon Kavramları

Kavram	Açıklama	Örnek
Fonksiyon çağırma	Fonksiyonun adını yazıp yanına parantez koyarak çalıştırmak.	`print()`
Argüman	Fonksiyonun parantezi içine verilen değer. Birden fazlaysa virgülle ayrılır.	`round(3.14159, 2)`
Dönüş değeri	Fonksiyonun işini bitirince geri verdiği sonuç. Bir değişkene atanabilir.	`sonuc = round(3.14159, 2)`

```python
sonuc = round(3.14159, 2)   # round fonksiyonu çağrıldı; 3.14159 ve 2 birer argüman
print(sonuc)                # 3.14
```

> **Dikkat:** Parantez olmadan yazılan `print` gibi bir fonksiyon ifadesi fonksiyonu çalıştırmaz. Fonksiyonun **çalışması** için `print()` biçiminde parantezle çağrılması gerekir.

19.1. Yerleşik (Built-in) Fonksiyonlar

Python'la birlikte hazır gelen fonksiyonlara **yerleşik fonksiyon** denir. Bu konuda göreceğimiz `print()` ve `input()` ile tür dönüşümünde kullandığımız `int()`, `float()` ve `str()` yerleşik fonksiyonlardır.

Sık Kullanılan Yerleşik Fonksiyonlar

Fonksiyon	Ne yapar?	Örnek	Sonuç
`print()`	Değerleri ekrana yazdırır.	`print("Merhaba")`	Ekrana `Merhaba` yazar
`input()`	Kullanıcıdan klavyeyle veri alır.	`input("Adınız: ")`	Yazılan metin
`int()`	Değeri tam sayıya çevirir.	`int("25")`	`25`
`float()`	Değeri ondalıklı sayıya çevirir.	`float("2.5")`	`2.5`
`str()`	Değeri metne çevirir.	`str(15)`	`"15"`
`type()`	Değerin türünü verir.	`type(5)`	`<class 'int'>`
`len()`	Metnin karakter sayısını verir.	`len("Python")`	`6`
`round()`	Sayıyı yuvarlar.	`round(3.14159, 2)`	`3.14`
`abs()`	Mutlak değer verir.	`abs(-7)`	`7`
`max()` / `min()`	En büyük / en küçük değeri verir.	`max(3, 9, 5)`	`9`

Birkaç yerleşik fonksiyonun sonuçları.

```
print(len("Python"))
print(max(3, 9, 5))
print(min(3, 9, 5))
print(abs(-7))
print(round(3.14159, 2))
print(type(5))
print(type("5"))
```

Ekran Çıktısı

```
6
9
3
7
3.14
`<class 'int'>`
`<class 'str'>`
```

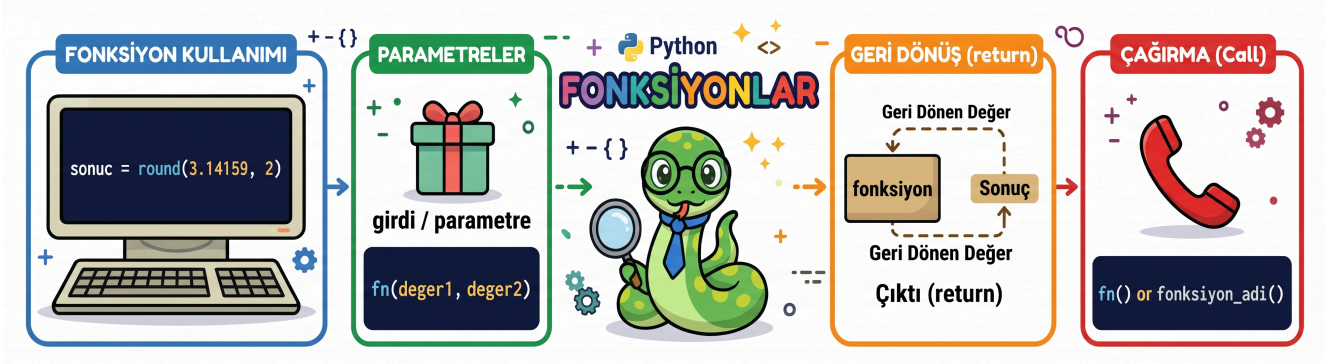
19.1.1. Dönüş Değeri Olan ve Olmayan Fonksiyonlar

`round()` gibi fonksiyonlar bir sonuç **geri verir** ve bu sonucu bir değişkene atayabiliriz. `print()` ise sonuç üretmez, işi ekrana yazmaktır. Bu yüzden `print()` çağrısının sonucunu bir değişkene atarsak `None` (yani "değer yok") elde ederiz.

```
a = round(2.678, 1)
print(a)          # 2.7

b = print("Merhaba")    # ekrana Merhaba yazılır
print(b)                # None
```

> **Ders notu:** Bir fonksiyonun ekrana yazdığı şey ile geri verdiği değer farklı kavramlardır. Ekrandaki yazıyı değişkende saklayamayız; ancak dönüş değerini saklayabiliriz.



20. input() ve print() Fonksiyonları

Çoğu program şu üç adımı izler: **veriyi al** (girdi), **işle**, **sonucu göster** (çıktı). `input()` girdi adımını, `print()` çıktı adımını yapar.

20.1. print(): Ekrana Yazdırma

`print()` fonksiyonu, parantez içine verilen değerleri ekrana yazar ve ardından bir alt satıra geçer. Metinler tırnak içinde yazılır; sayılar ve değişkenler tırnaksız yazılır.

Metin, sayı ve değişken yazdırma

```
print("Merhaba Dünya")
print(2026)
print(7 + 3)

isim = "Ayşe"
print(isim)
print("isim")
```

Ekran Çıktısı

```
Merhaba Dünya
2026
10
Ayşe
isim
```

> **Dikkat:** `print(isim)` değişkenin içindeki değeri (`Ayşe`) yazdırır. `print("isim")` ise tırnak içindeki metni (`isim`) olduğu gibi yazdırır.

Boş `print()` çağrısı ekrana hiçbir şey yazmaz, yalnızca bir satır boşluk bırakır:

```
print("Birinci satır") # Birinci satır
print()                #
print("Üçüncü satır")  # Üçüncü satır
```

20.2. input(): Kullanıcıdan Veri Alma

`input()` fonksiyonu program çalışırken klavyeden veri almamızı sağlar. Arka planda üç aşama gerçekleşir:

1. Python `input()` satırına gelince programın akışı **durur**.
2. Kullanıcı klavyeden bir şey yazıp **Enter** tuşuna basana kadar program bekler.
3. Enter'a basılınca yazılan her şey bir **metin (str)** olarak geri verilir ve sol taraftaki değişkene atanır. Program bir alt satırdan devam eder.

```
ad = input("Adınız nedir? ")
print("Merhaba", ad)
```

Parantez içine yazdığımız metin (`"Adınız nedir? "`) kullanıcıya gösterilen **yönlendirme mesajıdır**. Sonuna bir boşluk bırakmak, kullanıcının yazdığı cevabın mesaja yapışmamasını sağlar.

Kullanıcının adını alıp selamlama

```
ad = input("Adınız nedir? ")
print("Merhaba", ad)
print("Tanıştığımıza memnun oldum!")
```

Ekran Çıktısı

```
Adınız nedir? Ayşe
Merhaba Ayşe
Tanıştığımıza memnun oldum!
```

20.2.1. input() Her Zaman Metin Verir

Kullanıcı `15` yazsa bile `input()` bunu sayı olarak değil, `"15"` **metni** olarak verir. Bu, tür dönüşümü konusunda gördüğümüz önemli bir kuraldır:

input Her Zaman Metin Verir

```
yas = input("Yaşınız: ")
print(type(yas))
print(yas + yas)
```

Ekran Çıktısı

```
Yaşınız: 15
`<class 'str'>`
1515
```

Metinlerde `+` toplama değil **birleştirme** yaptığı için `"15" + "15"` sonucu `1515` olur. Sayısal işlem yapacaksak `int()` veya `float()` ile dönüştürmemiz gerekir:

Doğum yılından yaş hesaplama

```
dogum_yili = int(input("Doğum yılınız: "))
yas = 2026 - dogum_yili
print("Yaşınız:", yas)
```

Ekran Çıktısı

```
Doğum yılınız: 2010
Yaşınız: 16
```

Girilen Veriye Göre Dönüştürme

Beklenen veri	Kullanılacak yazım	Örnek
Metin (ad, şehir, mesaj)	`input("...")`	`sehir = input("Şehir: ")`
Tam sayı (yaş, adet)	`int(input("..."))`	`adet = int(input("Adet: "))`
Ondalıklı sayı (not, fiyat, boy)	`float(input("..."))`	`boy = float(input("Boy (m): "))`

> **Dikkat:** Kullanıcı `abc` gibi sayıya çevrilemeyecek bir şey yazarsa `int()` veya `float()` hata verir (`ValueError`) ve program durur. Hata yönetimini ileride göreceğiz; şimdilik kullanıcıdan doğru türde veri girildiğini varsayıyoruz.

20.3. print() ile Metin ve Değişkenleri Birlikte Yazdırma

Gerçek programlarda çoğu zaman sabit metin ile değişken değerlerini aynı satırda göstermek isteriz. Örneğin `Merhaba Ayşe, 16 yaşındasın.` Bunu yapmanın üç yolu vardır.

20.3.1. Virgül (,) ile Yazdırma

`print()` içindeki argümanlar virgülle ayrılır. Python aralarına **otomatik bir boşluk** koyar ve her değeri kendi türüne dokunmadan yazar. Bu yüzden sayı ile metni karıştırmak sorun çıkarmaz.

```
ad = "Ayşe"
yas = 16
print("Merhaba", ad, "yaşın", yas)   # Merhaba Ayşe yaşın 16
```

20.3.2. f-string (Formatlı Metin)

Modern ve en okunaklı yöntemdir. Metnin tırnağının hemen önüne **`f`** harfi yazılır; metnin içinde değişkenler **süslü parantez `{ }`** arasına konur. Python çalışırken süslü parantezin içindeki değişkeni değeriyle değiştirir.

Aynı çıktının f-string ile yazılışı

```
ad = "Ayşe"
yas = 16
print(f"Merhaba {ad}, {yas} yaşındasın.")
```

Ekran Çıktısı

```
Merhaba Ayşe, 16 yaşındasın.
```

f-string'in avantajları:

- Boşlukları ve tırnakları tek tek düşünmeyiz; cümle nasıl görünecekse öyle yazarız.
- Kod, okunduğunda yazdığımız cümleye çok benzer.

```
# Aynı iş, üç farklı yazım
ad = "Ayşe"
yas = 16

print("Merhaba", ad + ",", yas, "yaşındasın.")        # virgül ve + karışık
print(f"Merhaba {ad}, {yas} yaşındasın.")             # f-string
```

20.3.3. f-string İçinde İşlem Yapmak

Süslü parantezin içine yalnızca değişken değil, **her türlü ifade** yazabiliriz.

```
a = 8
b = 5
ad = "ayşe"

print(f"{a} + {b} = {a + b}")
print(f"{a} * {b} = {a * b}")
print(f"Ad: {ad.upper()}")
print(f"Ad {len(ad)} harflidir.")
```

Süslü parantez içinde ifade kullanımı:

Ekran Çıktısı

```
8 + 5 = 13
8 \* 5 = 40
8 sayısı çift mi? True
Ad: AYŞE
Ad 4 harflidir.
```

20.3.4. f-string ile Sayıları Biçimlendirme

Süslü parantezin içinde değişkenin yanına : (iki nokta) koyup nasıl gösterileceğini belirtebiliriz. En çok kullanılanlar:

f-string Biçimlendirme Kalıpları

Kalıp	Örnek	Sonuç	Açıklama
`:.2f`	`f"{3.14159:.2f}"`	`3.14`	Virgülden sonra 2 basamak göster (yuvarlar).
`:.0f`	`f"{2.5678:.0f}"`	`3`	Virgülden sonra basamak gösterme.
`:,`	`f"{1234567:,}"`	`1,234,567`	Binlik ayırıcı koy.
`:,.2f`	`f"{1234567.891:,.2f}"`	`1,234,567.89`	Binlik ayırıcı ve 2 basamak.
`:.1%`	`f"{0.256:.1%}"`	`25.6%`	Yüzde olarak göster.
`:03d`	`f"{7:03d}"`	`007`	Toplam 3 basamak olacak şekilde başa sıfır ekle.

```
fiyat = 249.9
adet = 3
toplam = fiyat * adet

print(f"Toplam: {toplam}")
print(f"Toplam: {toplam:.2f} TL")
print(f"Toplam: {toplam:,.2f} TL")
```

Fiyat hesabında biçimlendirme

Ekran Çıktısı

```
Toplam: 749.7
Toplam: 749.70 TL
Toplam: 749.70 TL
```

> **İpucu:** Para, ortalama, ölçüm gibi değerlerde `{deger:.2f}` kalıbı çok kullanılır. `0.1 + 0.2` gibi işlemlerde çıkan `0.30000000000000004` türü uzun sayıları da `:.2f` ile düzgün gösterebiliriz.

20.3.5. f-string ile Hizalama

Tablo gibi düzenli çıktılar için metni belirli bir genişlikte sola (<), sağa (>) veya ortaya (^) hizalar.

f-string Hizalama Kalıpları

Kalıp	Örnek	Sonuç (| işaretleri sınırı gösterir)	Açıklama
`:<10`	`f"{'Elma':<10}"`	`|Elma      |`	10 karakterlik alanda sola yasla.
`:>10`	`f"{'Elma':>10}"`	`|      Elma|`	10 karakterlik alanda sağa yasla.
`:^10`	`f"{'Elma':^10}"`	`|   Elma   |`	10 karakterlik alanda ortala.

Hizalı fiyat listesi

```
print(f"{'Ürün':<10}{'Fiyat':>8}")
print(f"{'Elma':<10}{12.5:>8.2f}")
print(f"{'Armut':<10}{9.75:>8.2f}")
print(f"{'Muz':<10}{30:>8.2f}")
```

Ekran Çıktısı

```
Ürün         Fiyat
Elma         12.50
Armut         9.75
Muz          30.00
```

20.3.6. f-string'de Süslü Parantez ve Tırnak Kullanımı

- Metnin içinde süslü parantezin kendisini göstermek istersek **iki kez** yazarız: `{{` ve `}}`.
- `{ }` içine doğrudan bir yazı yazacaksak, f-string'in dışındaki tırnaktan **farklı** tür tırnak kullanırız. Dışta çift tırnak varsa içeride tek tırnak kullanırız.

```
print(f"Küme: {{1, 2, 3}}")        # Küme: {1, 2, 3}

print(f"Ders: {'Python'}")         # Ders: Python
```

20.3.7. f"{degisken=}" Kullanımı

Süslü parantez içine değişkenin sonuna `=` koyarsak Python hem değişkenin **adını** hem **değerini** yazar. Kodu test ederken çok işe yarar.

```
puan = 75
ort = 82.5
print(f"{puan=}")
print(f"{ort=}")
```

Ekran Çıktısı

```
puan=75
ort=82.5
```

20.4. print() Fonksiyonunun sep ve end Parametreleri

`print()` fonksiyonunun, biz yazmasak da varsayılan değerlerle çalışan iki isteğe bağlı parametresi vardır. Bunları yazarak çıktının biçimini değiştirebiliriz.

print() Parametreleri

Parametre	Varsayılan değer	Ne işe yarar?
`sep` (separator)	`" "` (bir boşluk)	Virgülle ayrılan değerlerin arasına ne konacağını belirler.
`end`	`"\n"` (alt satıra geç)	Yazdırma bittikten sonra satırın nasıl sonlanacağını belirler.

20.4.1. sep: Değerler Arasına Ne Gelsin?

Farklı ayırıcılarla yazdırma

```
print("Pazartesi", "Salı", "Çarşamba")
print("Pazartesi", "Salı", "Çarşamba", sep="-")
print("Pazartesi", "Salı", "Çarşamba", sep=", ")
print("Pazartesi", "Salı", "Çarşamba", sep="")
print(14, 30, 45, sep=":")
```

Ekran Çıktısı

```
Pazartesi Salı Çarşamba
Pazartesi\-Salı\-Çarşamba
Pazartesi, Salı, Çarşamba
PazartesiSalıÇarşamba
14:30:45
```

20.4.2. end: Satır Nasıl Bitsin?

Normalde her `print()` işini bitirince alt satıra geçer. `end` ile bu davranışı değiştirebiliriz.

```
print("Yükleniyor", end="...")
print("Tamam")

print("A", end=" ")
print("B", end=" ")
print("C")
```

Aynı satırda yazdırmaya devam etme

Ekran Çıktısı

```
Yükleniyor...Tamam
A B C
```

`sep` ve `end` birlikte de kullanılabilir:

```
print("Gün", "Ay", "Yıl", sep="/", end=" <- tarih biçimi\n")
# Gün/Ay/Yıl <- tarih biçimi
```

20.5. Kaçış Karakterleri (Escape Characters)

Metin içinde yeni satır, sekme (tab) veya tırnak işareti gibi özel karakterler kullanmak için ters eğik çizgi `\` ile başlayan **kaçış karakterleri** kullanılır.

Sık Kullanılan Kaçış Karakterleri

Karakter	Anlamı	Örnek
`\n`	Yeni satıra geç.	`print("A\nB")`
`\t`	Sekme (tab) boşluğu bırak.	`print("A\tB")`
`\\`	Ters eğik çizgi karakterini yazdır.	`print("C:\\Okul")`
`\"`	Çift tırnak karakterini yazdır.	`print("O \"evet\" dedi")`
`\'`	Tek tırnak karakterini yazdır.	`print('Ali\'nin kitabı')`

Kaçış Karakteri Örneği

```
print("Birinci\nİkinci")
print("Ad:\tAyşe")
print("Yol: C:\\Okul")
print("O \"evet\" dedi")
```

Ekran Çıktısı

```
Birinci
İkinci
Ad:#h(1.5em)Ayşe
Yol: C:\Okul
O \"evet\" dedi
```

20.6. Örnek Uygulamalar

Daire alanı hesaplama

```
pi = 3.14159
r = float(input("Yarıçap (cm): "))
alan = pi * r * r
print(f"r={r}cm olan dairenin alanı {alan:.2f}cm²'dir.")
```

Ekran Çıktısı

```
Yarıçap (cm): 5 \
r=5.0 cm olan dairenin alanı
78.54 cm²\'dir.
```

Not ortalaması

```
ad = input("Öğrenci adı: ")
vize = float(input("Vize notu: "))
final = float(input("Final notu: "))

ortalama = vize * 0.4 + final * 0.6

print(f"Öğrenci : {ad}")
print(f"Ortalama: {ortalama:.1f}")
print(f"Geçti mi: {ortalama >= 50}")
```

Ekran Çıktısı

```
Öğrenci adı: Ayşe
Vize notu: 70
Final notu: 90
Öğrenci : Ayşe
Ortalama: 82.0
Geçti mi: True
```

Market fişi

```
urun = input("Ürün adı: ")
fiyat = float(input("Birim fiyat: "))
adet = int(input("Adet: "))

toplam = fiyat * adet

print("-" * 24)
print(f"{urun:<12}{adet:>3} x {fiyat:>6.2f}")
print(f"{'TOPLAM':<12}{toplam:>11.2f} TL")
print("-" * 24)
```

Ekran Çıktısı

```
Ürün adı: Defter
Birim fiyat: 24.5
Adet: 4
------------------------
Defter        4 x  24.50
TOPLAM            98.00 TL
------------------------
```

Saniyeyi saat-dakika-saniyeye çevirme

```
toplam = int(input("Saniye giriniz: "))
saat = toplam // 3600
kalan = toplam % 3600
dakika = kalan // 60
saniye = kalan % 60
print(f"{saat:02d}:{dakika:02d}:{saniye:02d}")
```

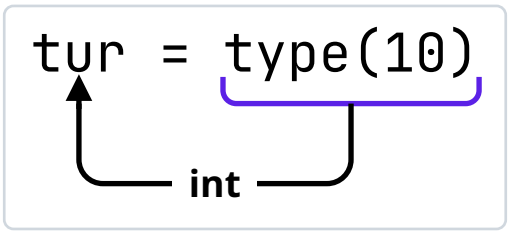
Ekran Çıktısı

```
Saniye giriniz: 3725
01:02:05
```

21. Python Tür Kontrolü: `type()` ve `isinstance()` Fonksiyonları

21.1. `type()` Fonksiyonu

Python'da her değerin bir türü vardır; **`type()`** fonksiyonu bu türü bize söyler. Python'da her değer (sayı, metin, liste, hatta fonksiyonlar) bir **nesnedir** ve her nesne bir **türe (sınıfa)** aittir. `type()` fonksiyonu, kendisine verilen nesnenin türünü döndürür.
Altaki şemada **`type(10)`** çağrısının sonucu **`type()`** fonksiyonu **`int`** sınıfını döndürerek **`tur`** değişkenine atıyor.



Temel kullanım

```
print(type(10))
print(type(3.14))
print(type("Merhaba"))
print(type(True))
```

Ekran Çıktısı

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Çıktıdaki **`<class 'int'>`** ifadesi şu anlama gelir: "Bu değer, **`int`** sınıfından bir nesnedir."

Sık Karşılaşılan Türler

Değer	type() sonucu	Açıklama
`42`	`int`	Tam sayı
`3.14`	`float`	Ondalıklı sayı
`"Python"`	`str`	Metin (string)
`True`	`bool`	Mantıksal değer
`[1, 2, 3]`	`list`	Liste
`(1, 2, 3)`	`tuple`	Demet
`{"a": 1}`	`dict`	Sözlük
`{1, 2, 3}`	`set`	Küme
`None`	`NoneType`	Değer yokluğu

Python **dinamik tipli** bir dildir. Bir değişkenin türünü önceden yazmayız; Python değere bakarak türü kendisi belirler. Üstelik aynı değişken sonradan başka türde bir değer alabilir.

Değişkenin türü değişebilir

```
x = 5
print(type(x))

x = "beş"
print(type(x))
```

Ekran Çıktısı

```
<class 'int'>
<class 'str'>
```

Tür önceden yazılmadığı için, bir değişkenin o anda hangi türde olduğunu görmek istediğimizde `type()` kullanırız. Hata ayıklarken (debug) en sık başvurulan araçlardan biridir.

`input()` Her Zaman `str` Döndürür

```
yas = input("Yaşınız: ")
print(type(yas))

yas = int(yas)
print(type(yas))
```

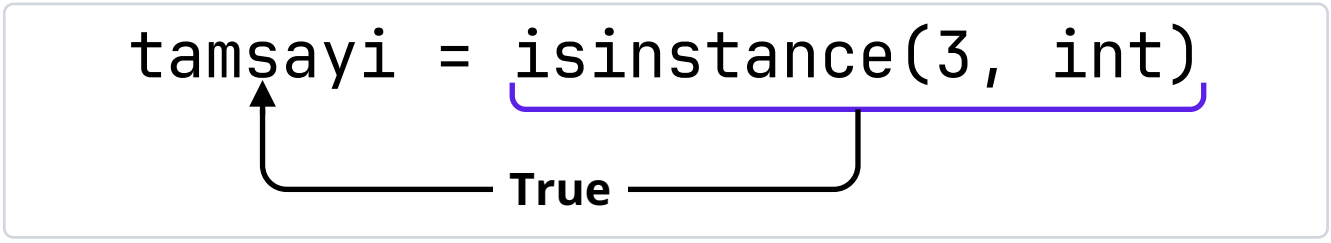
Ekran Çıktısı

```
Yaşınız: 15
<class 'str'>
<class 'int'>
```

> **Dikkat:** Kullanıcı `15` yazsa bile `input()` bunu `"15"` yani **metin** olarak verir. Matematiksel işlem yapmadan önce `int()` veya `float()` ile dönüştürmek gerekir.

21.2. `isinstance()` Fonksiyonu

Veri veya değişken türünü görmek için `type()` fonksiyonu kullanılır. Kullanıcının girdiği veri türünü kontrol etmek için **`isinstance()`** fonksiyonu çok kullanışlı bir araçtır.




```python
deger = 3
if isinstance(deger, int):
    print("Değer Bir Tamsayıdır!")

deger = 3.5
if isinstance(deger, float):
    print("Değer Kesirli Bir Sayıdır!")

if isinstance(deger, (int, float)):
    print("Değer Bir Sayıdır!")
```

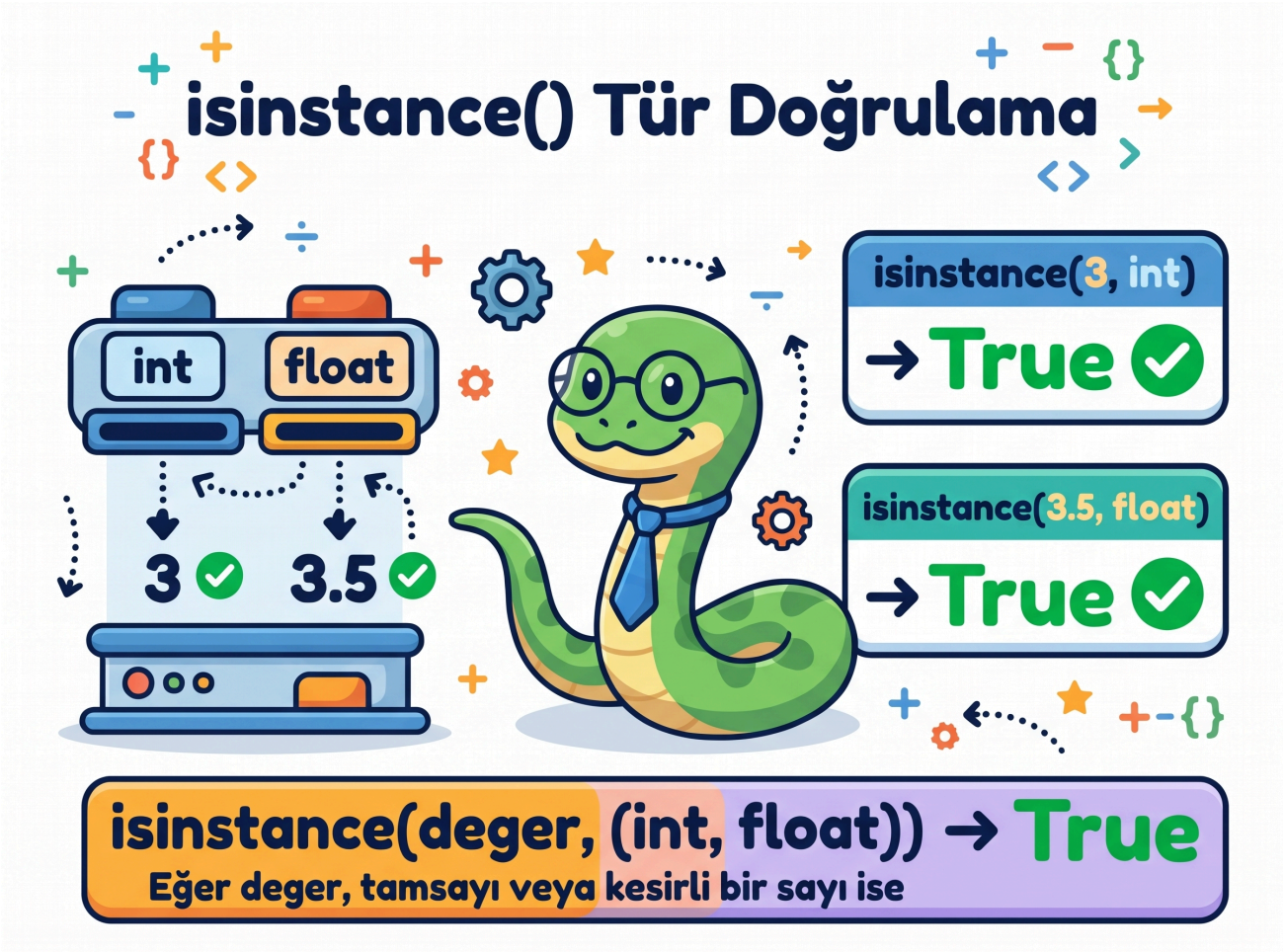
`isinstance()` Fonksiyonun Kullanımı

Ekran Çıktısı

```
Değer Bir Tamsayıdır!
Değer Kesirli Bir Sayıdır!
Değer Bir Sayıdır!
```

`isinstance()` fonksiyonu, değişkenlerin ve fonksiyon parametrelerinin tür kontrolü için oldukça uygundur. Aşağıdaki kodda `isinstance()` fonksiyonu, verilen değerin hem tam sayı (`int`) hem de ondalıklı sayı (`float`) olması durumunda `True` döndürecektir. Birden fazla tür kontrolü yapmanız gereken durumlar için `isinstance()` fonksiyonunu bu şekilde kullanabilirsiniz.

```python
isinstance(deger, (int, float))
```



22. Temel Programlama Yapıları Python Uygulama Soruları

Aşağıdaki sorular, temel programlama yapıları konusundaki bilgilere göre hazırlanmıştır. Her soruda kodun çalıştırıldığı ekran çıktısı verilmiş olup kullanıcının girdiği değerler örnek niteliğindedir. Siz de yazdığınız kodu kendi değerlerinizle deneyiniz.

22.1. Tanışma ve yaş hesabı

Konu: `input()`, `int()`, sabit, f-string · Zorluk: Kolay

Kullanıcıdan adını ve doğum yılını alan, yaşını hesaplayıp ekrana yazan bir program yazın. Programın en üstünde `BU_YIL` adında bir sabit (2026) tanımlayın ve çıktıyı f-string ile yazdırın.

Örnek Ekran Çıktısı

```
Adınız: Ayşe
Doğum yılınız: 2010
Merhaba Ayşe, 16 yaşındasın.
```

22.2. Dikdörtgenin alanı ve çevresi

Konu: `float()`, aritmetik operatörler, `:.2f` · Zorluk: Kolay

Kullanıcıdan bir dikdörtgenin kısa ve uzun kenar uzunluklarını (ondalıklı olabilir) alın. Alanı ve çevreyi hesaplayıp virgülden sonra 2 basamakla yazdırın.

Örnek Ekran Çıktısı

```
Kısa kenar (cm): 6.5
Uzun kenar (cm): 4
Alan: 26.00 cm²
Çevre: 21.00 cm
```

22.3. Not ortalaması

Konu: karşılaştırma operatörleri, `bool`, f-string · Zorluk: Kolay

Bir dersin ortalaması şöyle hesaplanıyor: 1. yazılı %30, 2. yazılı %30, performans %40. Üç notu kullanıcıdan alın; ortalamayı (virgülden sonra 1 basamak), dersi geçip geçmediğini (ortalama 50 veya üzeri) ve takdir alıp almadığını (ortalama 85 veya üzeri) yazdırın. Geçme ve takdir bilgisi `True` ya da `False` olarak görünmelidir.

Örnek Ekran Çıktısı

```
1. yazılı: 70
2. yazılı: 85
Performans notu: 90
Ortalama: 82.5
Geçti mi: True
Takdir mi: False
```

22.4. Tür dönüşümü

Konu: `type()`, `int()`, `float()`, otomatik dönüşüm · Zorluk: Kolay

Kullanıcıdan bir tam sayı alın ve aşağıdaki değerleri sırayla yazdıran bir program yazın. 1-4. ve 7. maddelerde değerin yanında `type()` ile türünü de gösterin.

1. Girilen değer, `input()`'tan geldiği haliyle
2. Aynı değerin tam sayıya çevrilmiş hali
3. Tam sayının ondalıklı sayıya çevrilmiş hali
4. `85.85` değerinin tam sayıya çevrilmiş hali
5. Girilen metnin kendisiyle toplamı (örneğin `25` için `2525`)
6. Tam sayının kendisiyle toplamı (örneğin `50`)
7. `adet = 5` ve `birim_fiyat = 2.5` değişkenlerinin çarpımı

Örnek Ekran Çıktısı

```
Bir sayı girin: 25
25 <class 'str'>
25 <class 'int'>
25.0 <class 'float'>
85 <class 'int'>
2525
50
12.5 <class 'float'>
```

22.5. Hatalı kodu düzeltin

Konu: değişken tanımlama kuralları, adlandırma, sabitler · Zorluk: Kolay

Aşağıdaki kodda değişken tanımlama kurallarına aykırı 5 satır vardır. Her hatanın nedenini yazın ve kodu `snake_case` yazımına uygun olacak şekilde düzeltin. Ardından `KDV_ORANI` (0.20) ve `MAKSIMUM_DENEME_SAYISI` (3) için sabit tanımlarını ekleyin. Düzeltilmiş program çalıştığında ekrana `100` yazmalıdır.

```
3etap_skor = 80
urun fiyati = 45.5
class = "9-A"
Toplam-Puan = 3etap_skor + 20
print(Toplam-Puan)
```

22.6. Kod izleme

Konu: işlem önceliği, bileşik atama, mantıksal operatörler · Zorluk: Orta

Aşağıdaki programı **çalıştırmadan**, her `print` satırının ekrana yazacağı değeri yanına yazın. Sonra bilgisayarda çalıştırıp kontrol edin.

```python
a = 10
b = 4

print(a + b * 2)
print((a + b) * 2)
print(a / b)
print(a // b)
print(a % b)
print(a > 5 and b < 3)
print(not a == 10 or b == 4)
```

```python
bakiye = 100
bakiye += 50
bakiye -= 30
bakiye *= 2
print(bakiye)

mesaj = "Ha"
mesaj *= 3
mesaj += "!"
print(mesaj)
```

```python
x = 7
x = x + 3 * 2

print(x)
print(x % 5 == 3)
print(17 // 5, 17 % 5)

print(2 ** 3 ** 2)
print(-3 ** 2)
print(10 - 4 - 3)
```

22.7. Çıktı biçimlendirme

Konu: `sep`, `end`, kaçış karakterleri · Zorluk: Orta

a) Kullanıcıdan gün, ay ve yılı ayrı ayrı alın. Tek bir `print()` ile, değerlerin arasına `/` koyarak ve satırın sonuna `<- tarih biçimi` yazarak ekrana yazdırın. Bunun için `sep` ve `end` kullanın.

Örnek Ekran Çıktısı

```
Gün: 14
Ay: 11
Yıl: 2026
14/11/2026 <- tarih biçimi
```

b) Aşağıdaki çıktıyı `print()` ile üreten programı yazın. "Yükleniyor... Tamam" aynı satırda görünmeli, diğer satırlarda kaçış karakterleri kullanılmalıdır (ikinci satırdaki boşluk bir `Tab` boşluğudur).

Ekran Çıktısı

```
Yükleniyor...Tamam
Ad: Ali
Yol: C:\Okul
O "evet" dedi
```

22.8. Dakikayı gün, saat ve dakikaya çevirme

Konu: tam bölme `//`, mod `%` · Zorluk: Orta

Kullanıcıdan toplam dakikayı alın; bu süreyi gün, saat ve dakika olarak yazdırın (1 gün = 1440 dakika). Yalnızca `//` ve `%` kullanın.

Örnek Ekran Çıktısı

```
Toplam dakika: 4000
2 gün 18 saat 40 dakika
```

22.9. Basamaklarına ayırma

Konu: tam bölme `//`, mod `%` · Zorluk: Orta

Kullanıcıdan 3 basamaklı bir sayı alın. Yüzler, onlar ve birler basamağını; basamakların toplamını; sayının tersini (487 için 784) ekrana yazdırın. Sayıyı metne çevirip dilimleme yapmayın, yalnızca `//` ve `%` kullanın.

Örnek Ekran Çıktısı

```
3 basamaklı bir sayı girin:
487
Yüzler: 4, Onlar: 8, Birler:
7
Basamaklar toplamı: 19
Ters sayı: 784
```

22.10. ATM para dağıtımı

Konu: `//` ve `%` ile adım adım hesap, değişken güncelleme · Zorluk: Zor

Bir ATM'de 100, 50, 20, 10, 5 ve 1 TL'lik paralar var. Kullanıcının çekmek istediği tutarı (tam sayı) alın ve her paradan en az sayıda kullanılarak kaç adet verileceğini yazdırın. İpucu: büyük paradan başlayın; her adımda `//` ile adedi, `%` ile kalanı bulun.

Örnek Ekran Çıktısı

```
Çekilecek tutar (TL): 287
100 TL: 2 adet
50 TL : 1 adet
20 TL : 1 adet
10 TL : 1 adet
5 TL  : 1 adet
1 TL  : 2 adet
```

22.11. Market fişi

Konu: sabit, f-string ile hizalama ve biçimlendirme · Zorluk: Zor

`KDV_ORANI = 0.20` sabitini tanımlayın. Kullanıcıdan ürün adını, birim fiyatı ve adedi alın. Ara toplamı, KDV'yi ve genel toplamı hesaplayın; aşağıdaki gibi, sütunları hizalı ve 28 tireli çizgilerle yazdırın. Ürün adı 12 karakterlik alanda sola, adet 3 karakterlik alanda sağa, fiyat 7 karakterlik alanda sağa yaslanmalıdır.

Örnek Ekran Çıktısı

```
Ürün adı: Defter
Birim fiyat: 24.5
Adet: 4
----------------------------
Defter        4 x   24.50
Ara toplam         98.00 TL
KDV (%20)          19.60 TL
GENEL TOPLAM      117.60 TL
----------------------------
```

22.12. Sinema bileti

Konu: karşılaştırma ve mantıksal operatörler · Zorluk: Orta

Bir sinema; 12 yaşından küçük, 65 yaş ve üzeri ya da öğrenci olanlara indirim yapıyor. Kullanıcıdan yaşını ve öğrenci olup olmadığını (`E` / `H`) alın; değişkenlerin adını `yas` ve `ogrenci` yapın. İndirim hakkını ve tam fiyat ödeyip ödemeyeceğini `True` / `False` olarak yazdırın. `if` kullanmayın; öğrenci bilgisini bir karşılaştırma ile `bool` değişkene dönüştürün. Bu iki değişkenin adını ve değerini f-string'in `{değişken=}` biçimiyle yazdırın. Son olarak, `0.1 + 0.2 == 0.3` ifadesinin sonucunu ve `round()` ile düzeltilmiş halini ekrana yazdırın.

Örnek Ekran Çıktısı

```
Yaşınız: 16
Öğrenci misiniz? (E/H): E
yas=16
ogrenci=True
İndirimli bilet: True
Tam fiyat: False
False
True
```

22.13. Üç sayının istatistiği

Konu: yerleşik fonksiyonlar `len`, `max`, `min`, `abs`, `round` · Zorluk: Orta

Kullanıcıdan ismini ve üç ondalıklı sayıyı alın. Yalnızca yerleşik fonksiyonları kullanarak isminin kaç harfli olduğunu, en büyük ve en küçük sayıyı, bu ikisinin farkını, üç sayının ortalamasını (2 basamağa yuvarlanmış) ve birinci ile ikinci sayının arasındaki mutlak farkı yazdırın.

Örnek Ekran Çıktısı

```
İsminiz: Mehmet
1. sayı: 7.5
2. sayı: 3
3. sayı: 9.25
İsminiz 6 harflidir.
En büyük: 9.25
En küçük: 3.0
Aralık: 6.25
Ortalama: 6.58
|1. sayı - 2. sayı| = 4.5
```

23. Python Uygulama Soruları: Cevap Anahtarı

Öğrencilerin değişken adları ve yazım biçimi farklı olabilir, çıktılar aynıysa doğru kabul edilebilir.

23.1. Tanışma ve yaş hesabı

```python
BU_YIL = 2026
ad = input("Adınız: ")
dogum_yili = int(input("Doğum yılınız: "))

yas = BU_YIL - dogum_yili

print(f"Merhaba {ad}, {yas} yaşındasın.")
```

Sık hata: `int()` kullanmayıp `"2026" - "2010"` işlemini yapmak `TypeError` hatası verir.

23.2. Dikdörtgenin alanı ve çevresi

```python
kisa_kenar = float(input("Kısa kenar (cm): "))
uzun_kenar = float(input("Uzun kenar (cm): "))

alan = kisa_kenar * uzun_kenar
cevre = 2 * (kisa_kenar + uzun_kenar)

print(f"Alan: {alan:.2f} cm²")
print(f"Çevre: {cevre:.2f} cm")
```

Sık hata: Çevre formülünde parantez unutmak (`2 * kisa + uzun` yanlış sonuç verir).

23.3. Not ortalaması

```python
yazili1 = float(input("1. yazılı: "))
yazili2 = float(input("2. yazılı: "))
perf = float(input("Performans notu: "))
ortalama = yazili1 * 0.3
ortalama += yazili2 * 0.3
ortalama += perf * 0.4

gecti_mi = ortalama >= 50
takdir_mi = ortalama >= 85

print(f"Ortalama: {ortalama:.1f}")
print(f"Geçti mi: {gecti_mi}")
print(f"Takdir mi: {takdir_mi}")
```

Sık hata: `gecti_mi = ortalama = 50` yazmak (karşılaştırma yerine atama).

23.4. Tür dönüşümü

```python
veri = input("Bir sayı girin: ")
print(veri, type(veri))

sayi = int(veri)
print(sayi, type(sayi))

ondalikli = float(sayi)
print(ondalikli, type(ondalikli))

kesilmis = int(85.85)
print(kesilmis, type(kesilmis))
```

```python
kesilmis = int(85.85)
print(kesilmis, type(kesilmis))

print(veri + veri)
print(sayi + sayi)

adet = 5
birim_fiyat = 2.5

toplam_tutar = adet * birim_fiyat
print(toplam_tutar, type(toplam_tutar))
```

Vurgulanacak noktalar: Metinle toplama birleştirme yapar (`2525`); `int` ile `float` çarpılınca sonuç `float` olur; `int(85.85)` yuvarlamaz, ondalık kısmı atar.

23.5. Hatalı kodu düzeltin

Satır	Hata nedeni	Düzeltme
`3etap_skor = 80`	Değişken adı rakamla başlayamaz	`etap3_skor`
`urun fiyati = 45.5`	Adın içinde boşluk olamaz	`urun_fiyati`
`class = "9-A"`	`class` Python'un anahtar kelimesidir	`sinif`
`Toplam-Puan = ...`	Tire (`-`) kullanılamaz; ayrıca `3etap_skor` adı hatalıdır	`toplam_puan = etap3_skor + 20`
`print(Toplam-Puan)`	Tire çıkarma işlemi gibi okunur	`print(toplam_puan)`

```python
etap3_skor = 80
urun_fiyati = 45.5
sinif = "9-A"
toplam_puan = etap3_skor + 20
```

```python
print(toplam_puan)

KDV_ORANI = 0.20
MAKSIMUM_DENEME_SAYISI = 3
```

Not: `class` bir anahtar kelime olduğu için değişken adı olamaz; yerine herhangi bir geçerli ad (`sinif`) kullanılabilir. Öğrenci her satırın hatasını doğru adlandırıyorsa tam puan verilebilir.

23.6. Kod izleme

Ekran Çıktısı

```
18
28
2.5
2
2
```

Ekran Çıktısı

```
512
-9
False
True
3
```

Ekran Çıktısı

```
240
HaHaHa!
13
True
3 2
```

Tartışılacak noktalar:

- `a + b * 2` ⟶ önce çarpma: `10 + 8 = 18`.
- `2 ** 3 ** 2` ⟶ üs alma sağdan sola: `2 ** 9 = 512`.
- `-3 ** 2` ⟶ üs alma eksi işaretinden önce: `-(3 ** 2) = -9`.
- `not a == 10 or b == 4` ⟶ karşılaştırmalar önce: `(not True) or True` = `True`.
- `bakiye`: 100 ⟶ 150 ⟶ 120 ⟶ 240.
- `x = 7 + 3 * 2` işleminin sonucu 13'tür; `13 % 5` 3 olduğundan sonuç `True`.

23.7. Çıktı biçimlendirme

```
gun = input("Gün: ")
ay = input("Ay: ")
yil = input("Yıl: ")

print(gun, ay, yil, sep="/", end=" <- tarih biçimi\n")
```

```
print("Yükleniyor", end="...")
print("Tamam")
print("Ad:\tAli")
print("Yol: C:\\Okul")
print("O \"evet\" dedi")
```

Not: `end` verilmezse her `print()` alt satıra geçer; `sep` yalnızca virgülle ayrılan değerlerin arasını etkiler. Ters eğik çizgiyi yazdırmak için `\\`, çift tırnağı yazdırmak için `\"` gerekir.

23.8. Dakikayı gün, saat ve dakikaya çevirme

```
toplam_dakika = int(input("Toplam dakika: "))

gun = toplam_dakika // 1440
kalan = toplam_dakika % 1440
saat = kalan // 60
dakika = kalan % 60

print(f"{gun} gün {saat} saat {dakika} dakika")
```

Kontrol: 4000 = 2 × 1440 + 1120 ve 1120 = 18 × 60 + 40. Ders notundaki "135 dakika → 2 saat 15 dakika" örneğinin bir adım genişletilmiş halidir.

23.9. Basamaklarına ayırma

```
sayi = int(input("3 basamaklı bir sayı girin: "))

yuzler = sayi // 100
onlar = (sayi // 10) % 10
birler = sayi % 10

basamak_toplami = yuzler + onlar + birler
ters_sayi = birler * 100 + onlar * 10 + yuzler

print(f"Yüzler: {yuzler}, Onlar: {onlar}, Birler: {birler}")
print(f"Basamaklar toplamı: {basamak_toplami}")
print(f"Ters sayı: {ters_sayi}")
```

Not: Onlar basamağı için önce `// 10` ile son basamak atılır (487 → 48), sonra `% 10` ile son basamak alınır (8). Bu, ders notundaki "son basamağı al / at" örneğinin devamıdır.

23.10. ATM para dağıtımı

```
tutar = int(input("Çekilecek tutar (TL): "))

yuzluk = tutar // 100
kalan = tutar % 100

ellilik = kalan // 50
kalan = kalan % 50

yirmilik = kalan // 20
kalan = kalan % 20

onluk = kalan // 10
kalan = kalan % 10

beslik = kalan // 5
birlik = kalan % 5

print(f"100 TL: {yuzluk} adet")

print(f"50 TL : {ellilik} adet")

print(f"20 TL : {yirmilik} adet")

print(f"10 TL : {onluk} adet")

print(f"5 TL  : {beslik} adet")
print(f"1 TL  : {birlik} adet")
```

Kontrol: 2×100 + 50 + 20 + 10 + 5 + 2×1 = 287. Bu soru, `kalan` değişkeninin her adımda yeniden atanarak güncellendiğini göstermek için iyi bir fırsattır.

23.11. Market fişi

```python
KDV_ORANI = 0.20

urun = input("Ürün adı:")
fiyat = float(input("Birim fiyat:"))
adet = int(input("Adet:"))

ara_toplam = fiyat * adet
kdv = ara_toplam * KDV_ORANI
genel_toplam = ara_toplam + kdv

print("-" * 28)

print(f"{urun:<12}{adet:>3} x {fiyat:>7.2f}")
print(f"{'Ara toplam':<14}{ara_toplam:>9.2f} TL")

print(f"{'KDV (%20)':<14}{kdv:>9.2f} TL")
print(f"{'GENEL TOPLAM':<14}{genel_toplam:>9.2f} TL")

print("-" * 28)
```

23.12. Sinema bileti

```python
yas = int(input("Yaşınız: "))
ogrenci = input("Öğrenci misiniz? (E/H): ") == "E"

indirimli = yas < 12 or yas >= 65 or ogrenci
tam_fiyat = not indirimli

print(f"{yas=}")
print(f"{ogrenci=}")
print(f"İndirimli bilet: {indirimli}")
print(f"Tam fiyat: {tam_fiyat}")
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 2) == 0.3)
```

Çıktı öğrencinin girdiği değerlere göre değişir. Üç farklı deneme:

Yaş	Öğrenci	İndirimli bilet	Tam fiyat
16	E	True	False
70	H	True	False
30	H	False	True

> **Not:** Son iki satır her zaman `False` ve `True` yazar: ondalıklı sayılar tam kesinlikle saklanamadığı için `0.1 + 0.2` ifadesi `0.30000000000000004` olur. Bu, ders notu 18.4'teki uyarının uygulamasıdır.

23.13. Üç sayının istatistiği

```python
isim = input("İsminiz: ")
a = float(input("1. sayı: "))
b = float(input("2. sayı: "))
c = float(input("3. sayı: "))

en_buyuk = max(a, b, c)
en_kucuk = min(a, b, c)
aralik = en_buyuk - en_kucuk
ortalama = round((a + b + c) / 3, 2)
fark = abs(a - b)

print(f"İsminiz {len(isim)} harflidir.")
print(f"En büyük: {en_buyuk}")
print(f"En küçük: {en_kucuk}")
print(f"Aralık: {aralik}")
print(f"Ortalama: {ortalama}")
print(f"|1. sayı - 2. sayı| = {fark}")
```

> **Not:** Girdiler `float()` ile çevrildiği için `3` değeri `3.0` olarak yazılır; öğrenciler bunu hata sanabilir.